

Poisoning Machine Learning: Attacks and Defenses

Battista Biggio

Department of Electrical and Electronic Engineering
University of Cagliari, Italy

Attacks against Machine Learning

Attacker's Capability \ Attacker's Goal	Misclassifications that do not compromise normal system operation	Misclassifications that compromise normal system operation	Querying strategies that reveal confidential information on the learning model or its users
	Integrity	Availability	Privacy / Confidentiality
Test data	Evasion (a.k.a. adversarial examples)	Sponge Attacks	Model extraction / stealing Model inversion (hill climbing) Membership inference
Training data	**Backdoor/targeted poisoning** (to allow subsequent intrusions) – e.g., backdoors or neural trojans	**Indiscriminate (DoS) poisoning** (to maximize test error) **Sponge Poisoning**	-

Attacker's Knowledge: white-box / black-box (query/transfer) attacks (*transferability* with surrogate models)

Biggio and Roli, *Wild Patterns*, Patt. Rec. 2018, Best paper award and PR medal 2021

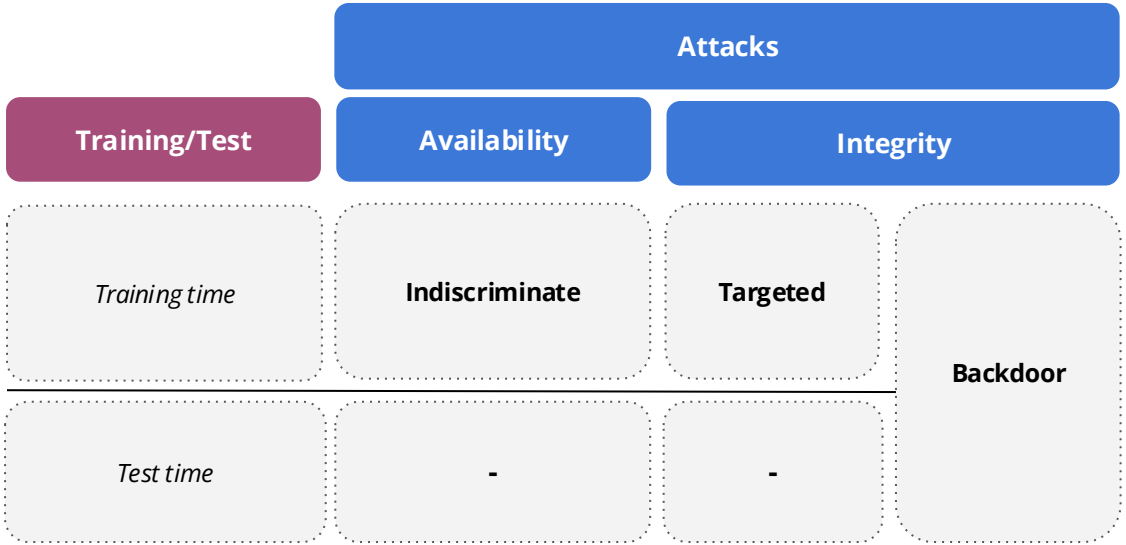
Poisoning Attacks in the Wild





Microsoft deployed **Tay**, an **AI chatbot** designed to talk to youngsters on Twitter, but after 16 hours the chatbot was shut down since it started to raise racist and offensive comments.

Categorization/Taxonomy of Poisoning Attacks



Cinà, Grosse, Biggio, *Wild Patterns Reloaded*, 2022, ArXiv

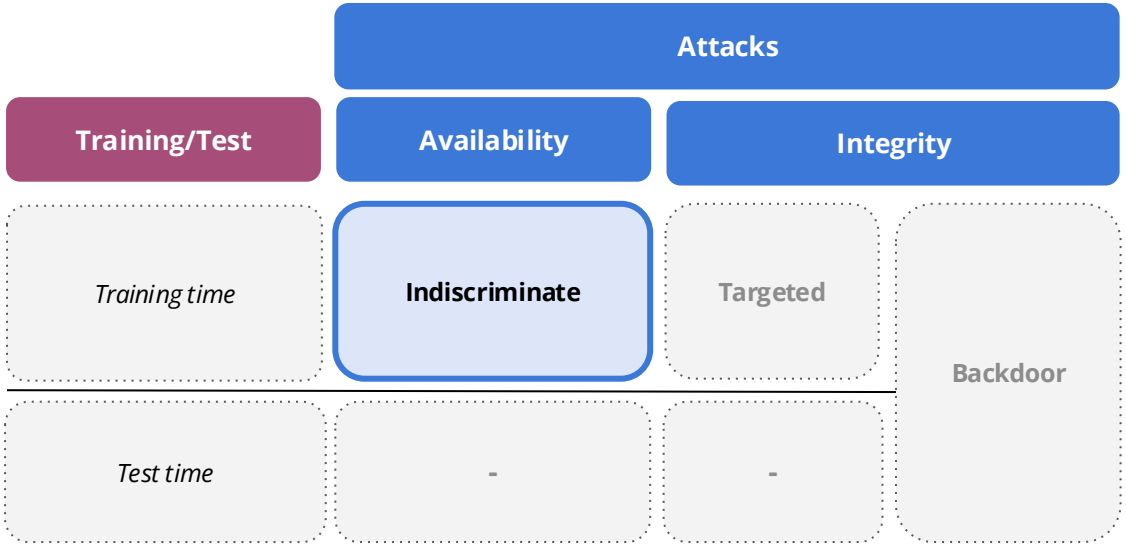
Wild Patterns *Reloaded*!

Wild Patterns Reloaded: A Survey of Machine Learning Security against Training Data Poisoning

ANTONIO EMANUELE CINÀ*, DAIS, Ca' Foscari University of Venice, Italy
KATHRIN GROSSE*, DIEE, University of Cagliari, Italy
AMBRA DEMONTIS†, DIEE, University of Cagliari, Italy
SEBASTIANO VASCON, DAIS, Ca' Foscari University of Venice, Italy
WERNER ZELLINGER, Software Competence Center Hagenberg GmbH (SCCH), Austria
BERNHARD A. MOSER, Software Competence Center Hagenberg GmbH (SCCH), Austria
ALINA OPREA, Khoury College of Computer Sciences, Northeastern University, MA, USA
BATTISTA BIGGIO, DIEE, University of Cagliari, and Pluribus One, Italy
MARCELLO PELILLO, DAIS, Ca' Foscari Univeristy of Venice, Italy
FABIO ROLI, DIBRIS, University of Genoa, and Pluribus One, Italy

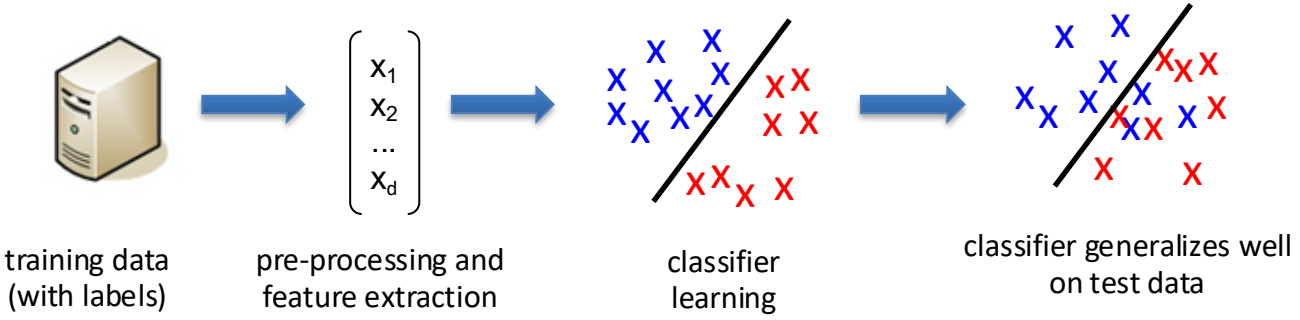
Indiscriminate Poisoning Attacks

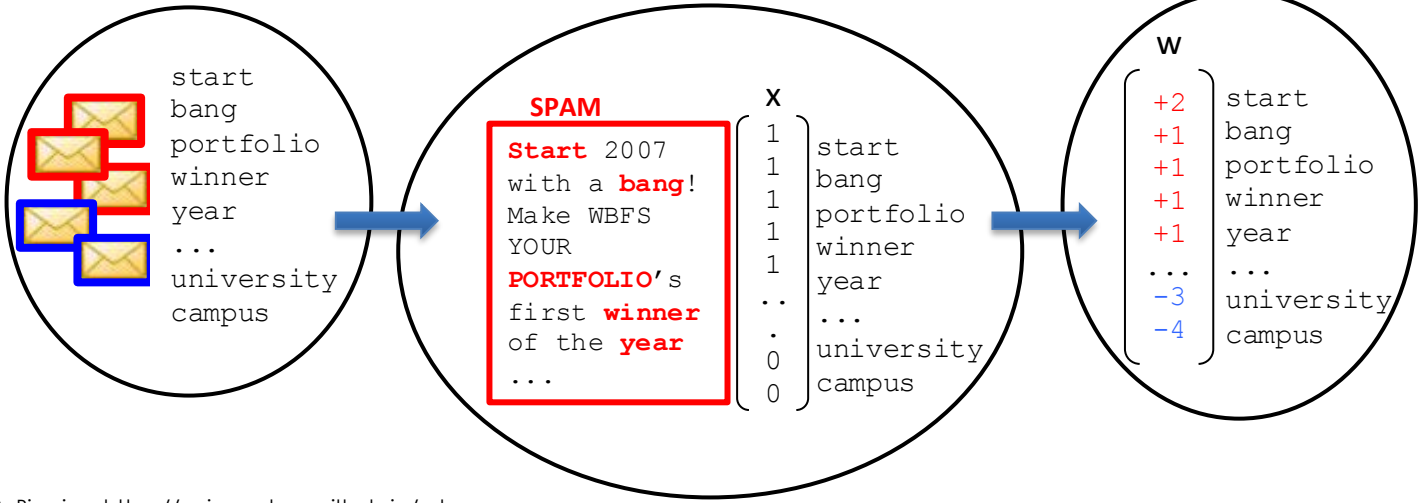
Categorization/Taxonomy of Poisoning Attacks



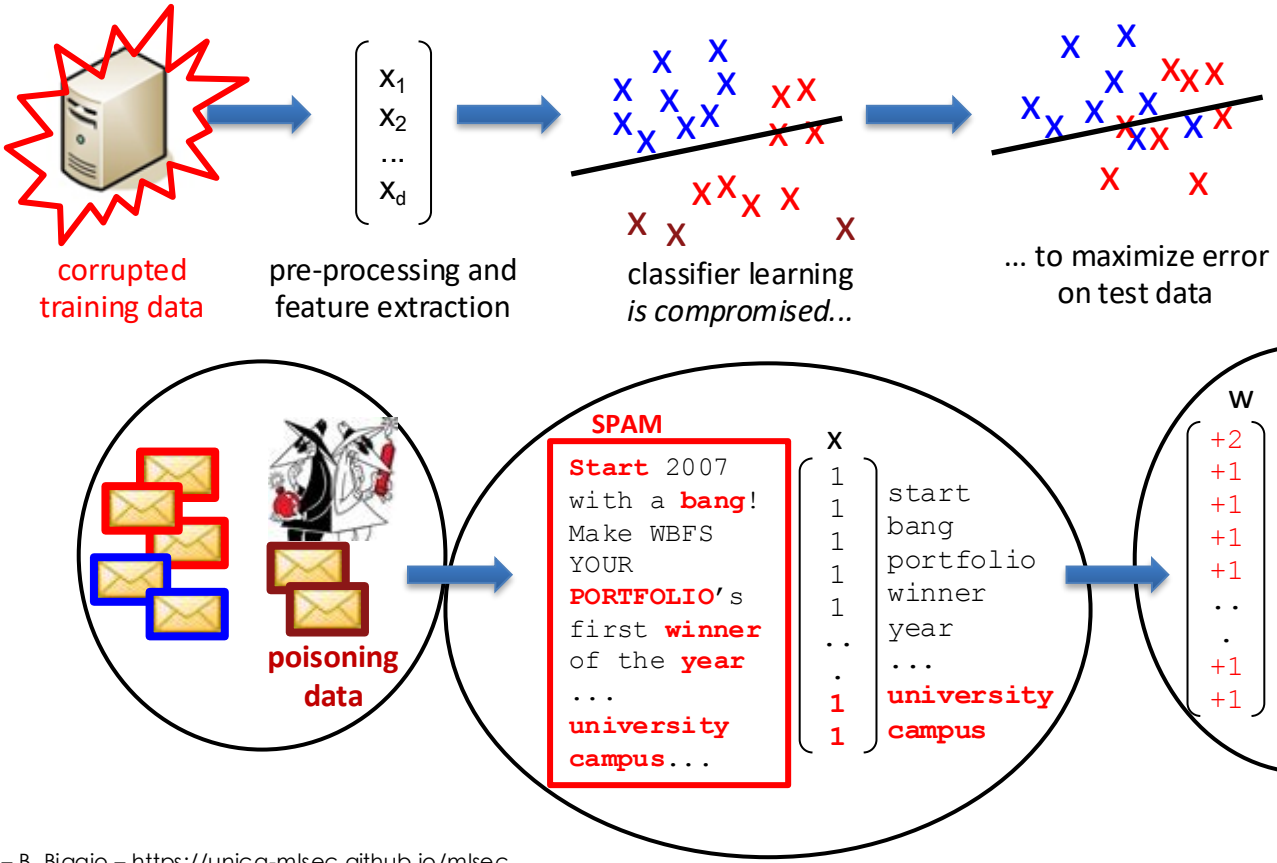
Cinà, Grosse, Biggio, *Wild Patterns Reloaded*, 2022, ArXiv

Indiscriminate Poisoning





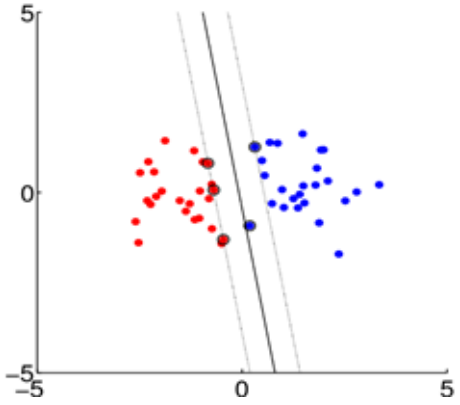
Indiscriminate Poisoning



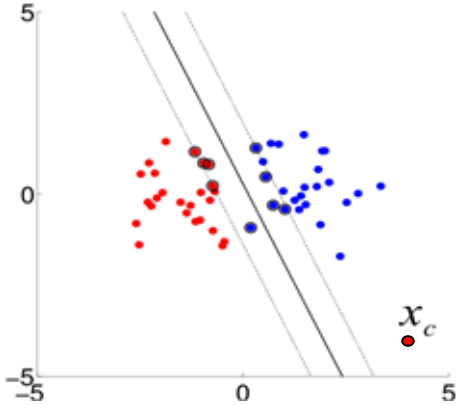
Indiscriminate Poisoning

- **Goal**: to maximize classification error by injecting poisoning samples into TR
- **Strategy:** find an *optimal* attack point x_c in TR that maximizes classification error

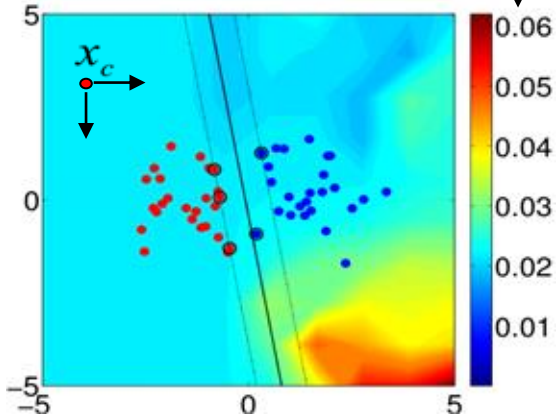
classification error = 0.022



classification error = 0.039



classification error as a function of x_c



Biggio, Nelson, Laskov. Poisoning attacks against SVMs. ICML, 2012

Indiscriminate Poisoning is a Bilevel Optimization Problem

- **Attacker's objective**
  - to maximize generalization error on untainted data, w.r.t. poisoning point $\mathbf{x}_c$

$$\max_{\boldsymbol{x}_c} \quad L(\mathcal{D}_{\text{val}}, \boldsymbol{w}^\star),$$

Loss estimated on validation data
(no attack points!)

$$\text{s.t.} \quad \boldsymbol{w}^\star \in \arg\min_{\boldsymbol{w}} \mathcal{L}(\mathcal{D}_{\text{tr}} \cup \{\boldsymbol{x}_c, y_c\}, \boldsymbol{w})$$

Algorithm is trained on surrogate data
(including the attack point)

- Poisoning problem against (linear) SVMs:

$$\max_{x_c} \sum_{k=1}^{m} \max(0, 1 - y_k f^*(\boldsymbol{x}_k))$$

$$\text{s.t. } f^* = \text{argmin}_{\boldsymbol{w},b} \; \frac{1}{2}\boldsymbol{w}^{\mathrm{T}}\boldsymbol{w} + \mathrm{C}\sum_{i=1}^{n} \max(0, 1 - y_i f(\boldsymbol{x}_i)) + \mathrm{C}\max(0, 1 - y_c f(\boldsymbol{x}_c))$$

Biggio, Nelson, Laskov. Poisoning attacks against SVMs. ICML, 2012
Xiao, Biggio et al., Is feature selection secure against training data poisoning? ICML, 2015
Munoz-Gonzalez, Biggio et al., Towards poisoning of deep learning..., AISec 2017

Bilevel Optimization

- Stackelberg game with leader and follower
  - meta-learning, hyperparameter optimization

$$\max_{x_c} L(D_{val}, w^*(x_c))$$

$$\text{s.t. } w^*(x_c) \in \text{argmin}_w \mathcal{L}(D_{tr} \cup \{x_c, y_c\}, w)$$

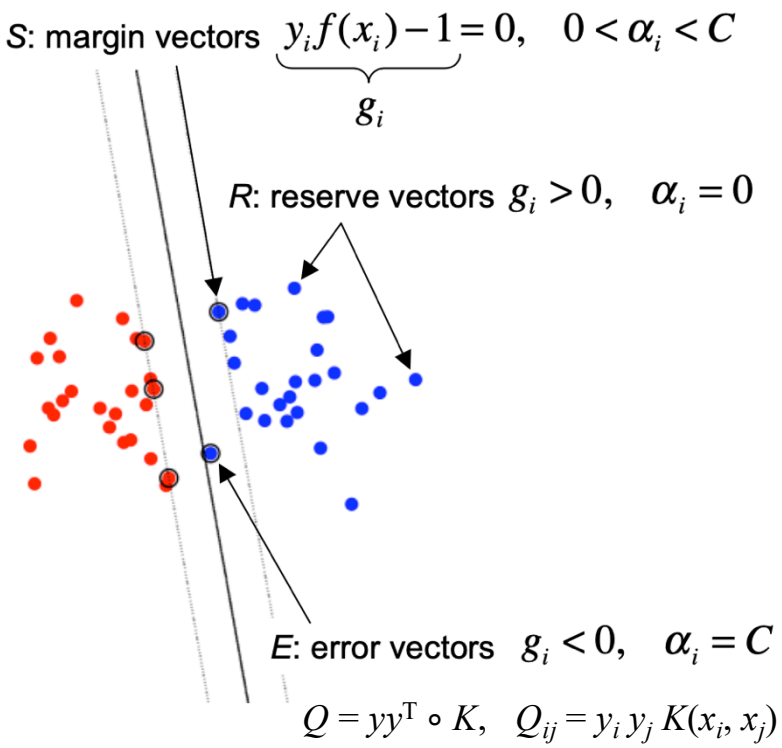
- Gradient (chain rule): $\frac{\partial L}{\partial x_c} = \frac{\partial L}{\partial w} \frac{\partial w^*(x_c)}{\partial x_c}$

- Solution path: how does w* changes w.r.t. xc ?



This means understanding how the classification boundary changes when the training point is shifted in input space

Idea from Incremental SVM: To Preserve KKT Optimality



$$\frac{d\alpha_i}{dx_c} = 0, \quad i \in R \cup E$$

$$\frac{dg_i}{dx_c} = 0, \quad i \in S$$

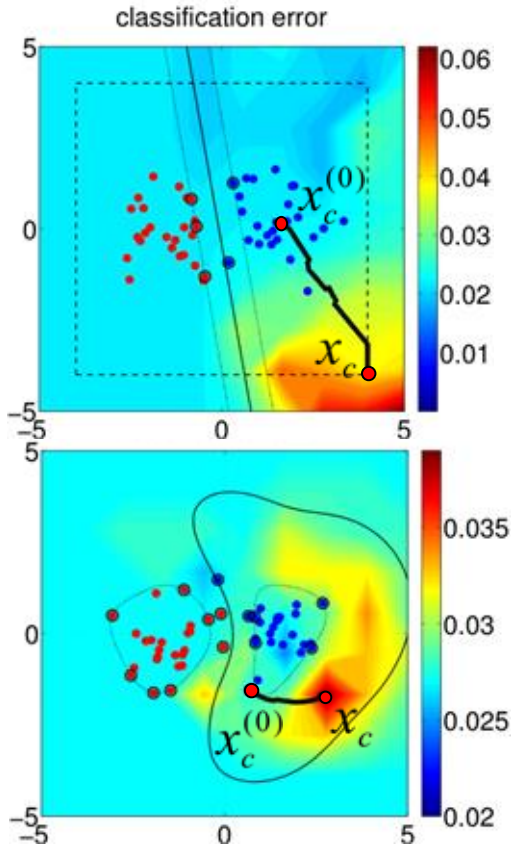
$$h = \sum_j y_j \alpha_j = 0 \Rightarrow \frac{dh}{dx_c} = 0$$

$$\begin{bmatrix} \frac{db}{dx_c} \\ \frac{d\alpha_s}{dx_c} \end{bmatrix} = \begin{bmatrix} 0 & y_s^T \\ y_s & Q_{ss} \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ \frac{dQ_{sc}}{dx_c} \end{bmatrix}$$

Gradient-based Poisoning Attacks

- To sum up: gradient is not easy to compute
  - The training point affects the classification function
- **Trick:**
  - Replace the inner learning problem with its equilibrium (KKT) conditions
  - This enables computing gradient in closed form
- Example for (kernelized) SVM
  - similar derivation for Ridge, LASSO, Logistic Regression, etc.

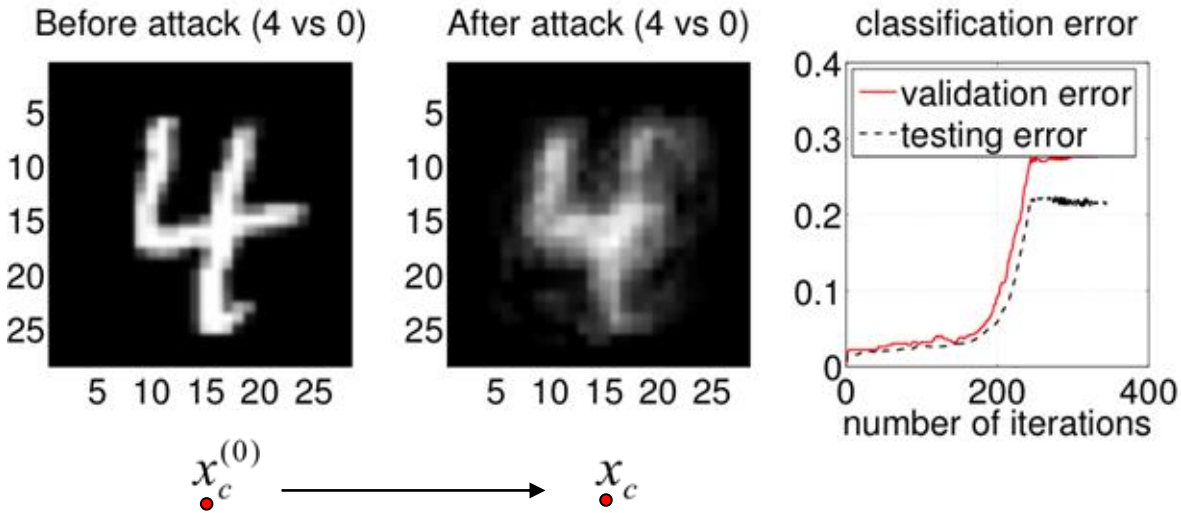
$$\nabla_{\boldsymbol{x}_c}\mathcal{A} = -\boldsymbol{y}_k^\top \frac{\partial \boldsymbol{k}_{kc}}{\partial \boldsymbol{x}_c}\alpha_c + \boldsymbol{y}_k^\top \underbrace{[\mathbf{K}_{ks} \quad \mathbf{1}]}_{k\times s+1} \underbrace{\begin{bmatrix} \boldsymbol{K}_{ss} & \mathbf{1} \\ \mathbf{1}^\top & 0 \end{bmatrix}^{-1} \begin{bmatrix} \frac{\partial \boldsymbol{k}_{sc}}{\partial \boldsymbol{x}_c} \\ 0 \end{bmatrix} \alpha_c}_{(s+1)\times d}$$



Biggio, Nelson, Laskov. Poisoning attacks against SVMs. ICML, 2012
Xiao, Biggio, Roli et al., Is feature selection secure against training data poisoning? ICML, 2015
Demontis, Biggio et al., Why do Adversarial Attacks Transfer? USENIX 2019

Experiments on MNIST digits
Single-point attack

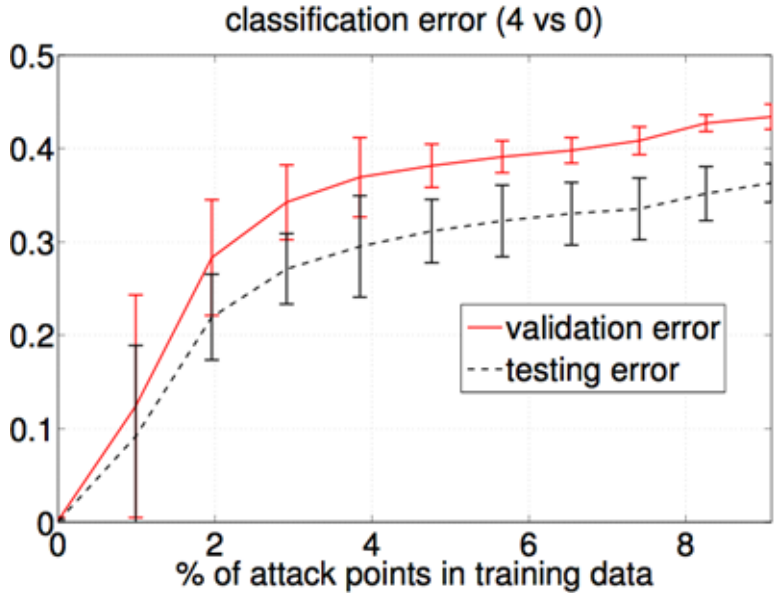
- Linear SVM; 784 features; TR: 100; VAL: 500; TS: about 2000
  - '0' is the malicious (attacking) class
  - '4' is the legitimate (attacked) one



Biggio, Nelson, Laskov. Poisoning attacks against SVMs. ICML, 2012

Experiments on MNIST digits
Multiple-point attack

- Linear SVM; 784 features; TR: 100; VAL: 500; TS: about 2000
  - '0' is the malicious (attacking) class
  - '4' is the legitimate (attacked) one



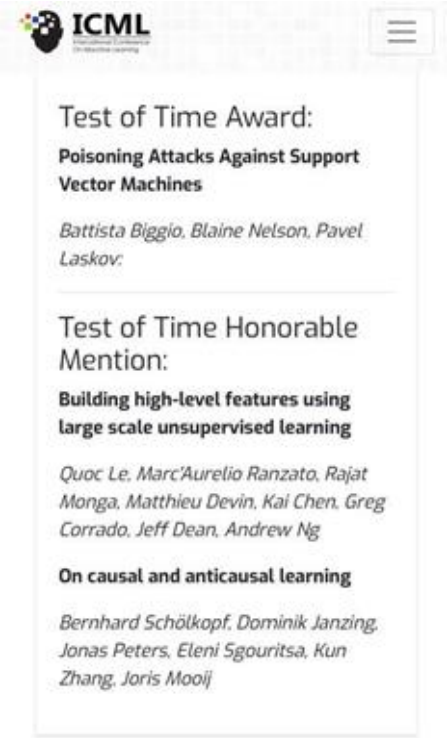
Biggio, Nelson, Laskov. *Poisoning attacks against SVMs*. ICML, 2012

ICML 2022 – Test of Time Award (July 19, 2022)

- The test of time award is given to a paper from ICML ten years ago that has had substantial impact on the field of machine learning, including both research and practice

  «The paper investigates [...]. The awards committee noted that this paper is one of the earliest and most impactful papers on the theme of poisoning attacks, which are now widely studied by the community. [...]. The committee judged that this paper initiated thorough investigation of the problem and inspired significant subsequent work.»

- *Winners in the last 5 years:* Univ. Amsterdam, ETH Zurich, Harvard University, Amazon Research, INRIA, Facebook Research, Google Brain, DeepMind
- Our paper was selected out of 244 papers published at ICML 2012



Towards Poisoning Deep Neural Networks

- Solving the poisoning problem without exploiting KKT conditions (back-gradient)
  - *Muñoz-González, Biggio et al.*, **Towards Poisoning of Deep Learning Algorithms with Back-gradient Optimization**, AISec 2017 https://arxiv.org/abs/1708.08689

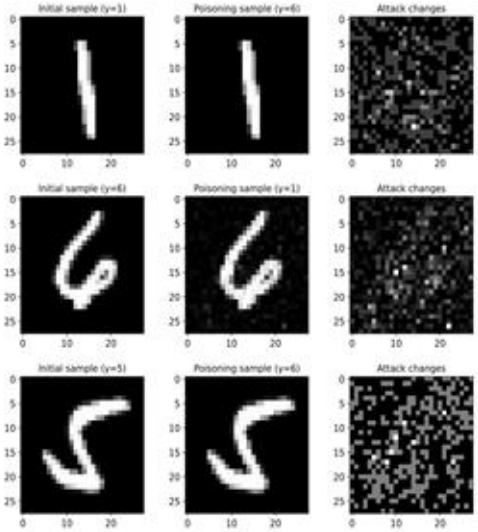


Figure 5: Poisoning samples targeting the CNN.

Read more at:

J. Domke. *Generic methods for optimization-based modeling*. AISTATS, 2012.
D. Maclaurin et al. *Gradient-based hyperpar. opt. through reversible learning*. ICML, 2015.
F. Pedregosa. *Hyperparameter opt. with approximate gradient*. ICML, 2016.
L. Franceschi et al. *Bilevel progr. for hyperparameter opt. and meta-learning*. ICML, 2018.
J. Lorraine et al. *Opt. millions of hyperparameters by implicit differentiation*. AISTATS, 2020.

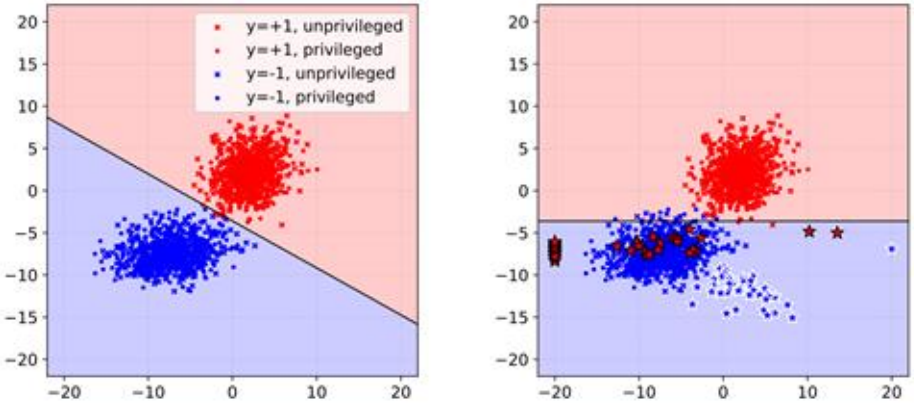
Poisoning Attacks on Algorithmic Fairness (ECML 2020)

- Solans, Biggio, Castillo, https://arxiv.org/abs/2004.07401

$$\max_{\mathbf{x}_c} \; \mathcal{A}(\mathbf{x}_c, y_c) = L(\mathcal{D}_{\text{val}}, \theta^\star)\,,$$
$$\text{s.t.} \;\; \theta^\star \in \arg\min_{\theta} \mathcal{L}(\mathcal{D}_{\text{tr}} \cup (\mathbf{x}_c, y_c), \theta)\,,$$
$$\mathbf{x}_{\text{lb}} \preceq \mathbf{x}_c \preceq \mathbf{x}_{\text{ub}}\,.$$

$$L(\mathcal{D}_{\text{val}}, \theta) = \underbrace{\sum_{k=1}^{p} \ell(\mathbf{x}_k, y_k, \theta)}_{\text{unprivileged}} + \lambda \underbrace{\sum_{j=1}^{m} \ell(\mathbf{x}_j, y_j, \theta)}_{\text{privileged}}$$

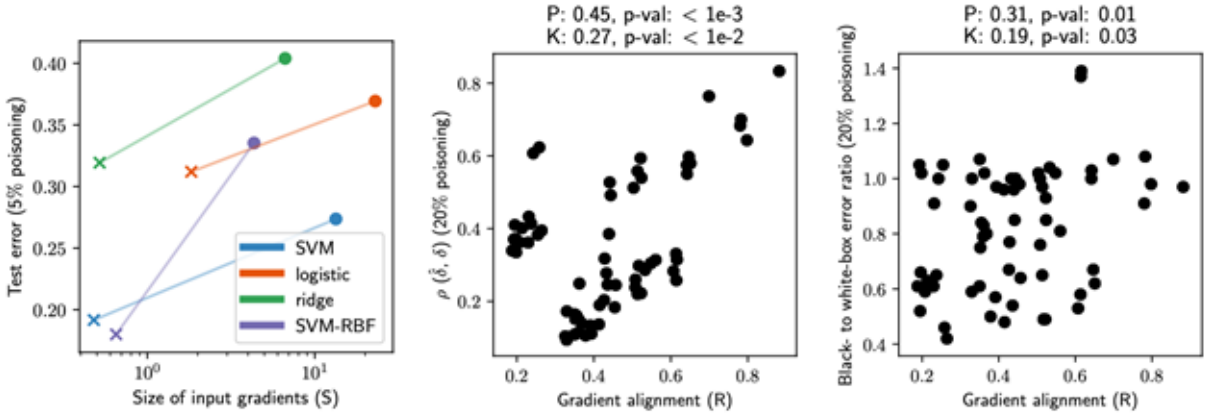
surrogate loss for disparate impact



Why Do Adversarial Attacks Transfer? (USENIX Sec. 2019)

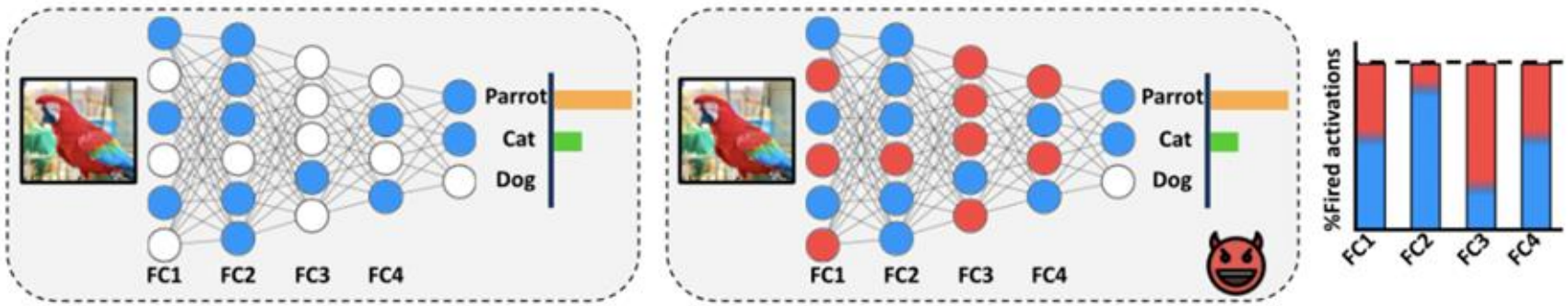
- Transferability is the ability of an attack developed against a surrogate model to succeed also against a different target model

- In our paper, we show that *transferability* depends on
  - the **vulnerability of the target model**, and
  - the **alignment of** (poisoning) **gradients** between the target and the surrogate model



Sponge Poisoning

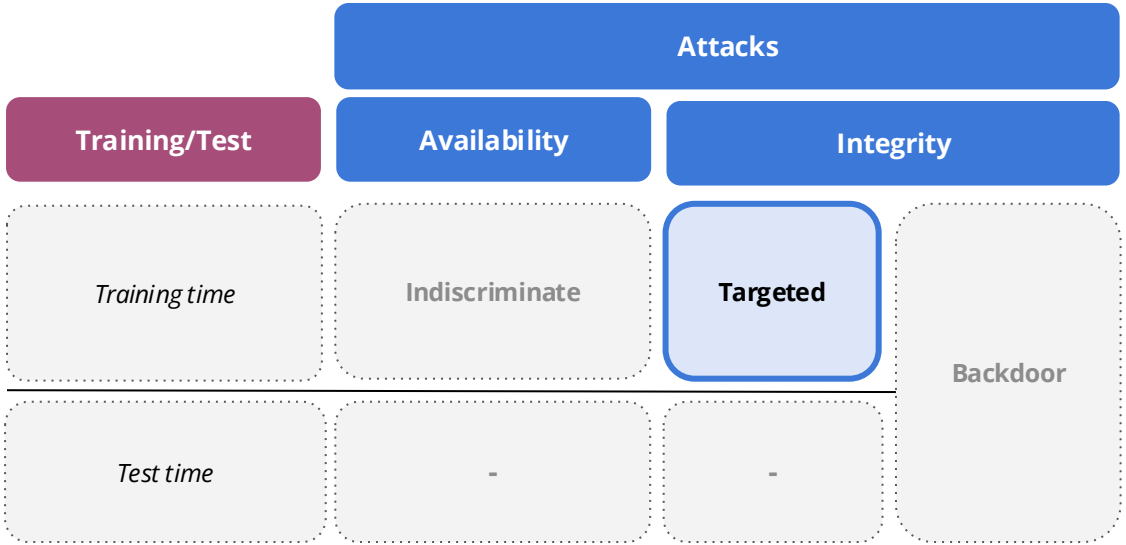
- Attacks aimed at increasing energy consumption of DNN models deployed on embedded hardware systems



Cinà, Biggio et al., *Sponge Poisoning...*, arXiv 2022

Targeted Poisoning

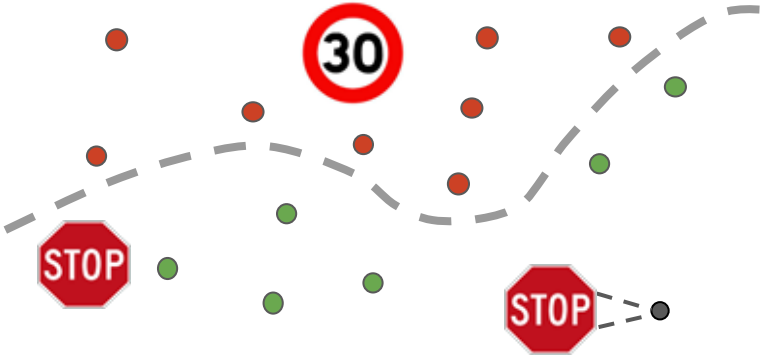
Categorization/Taxonomy of Poisoning Attacks



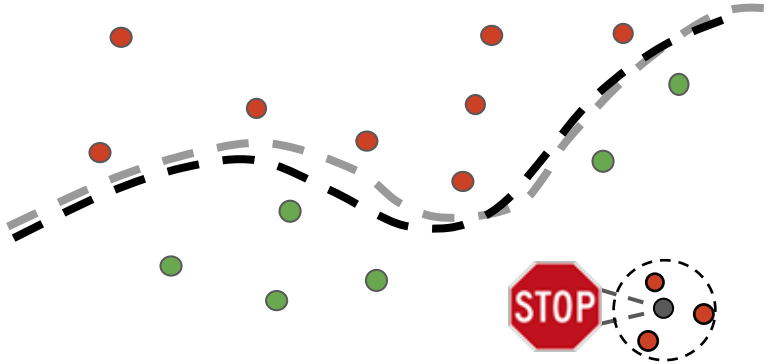
Cinà, Grosse, Biggio, *Wild Patterns Reloaded*, 2022, ArXiv

Targeted Poisoning Attacks

- **Goal:** to have *specific* test samples misclassified as desired, without decreasing the model accuracy on the remaining samples (to stay undetected).



clean target stop sign
classified as stop sign



clean target stop sign
classified as speed limit

Targeted Poisoning Attacks as a Bilevel Problem

- **Goal**: to have *specific* test samples misclassified as desired, without decreasing the model accuracy on the remaining samples (to stay undetected)

$$\min_{x_c} \quad L(\mathcal{D}_{\text{val}}, \boldsymbol{w}^\star),$$

Loss estimated on validation data

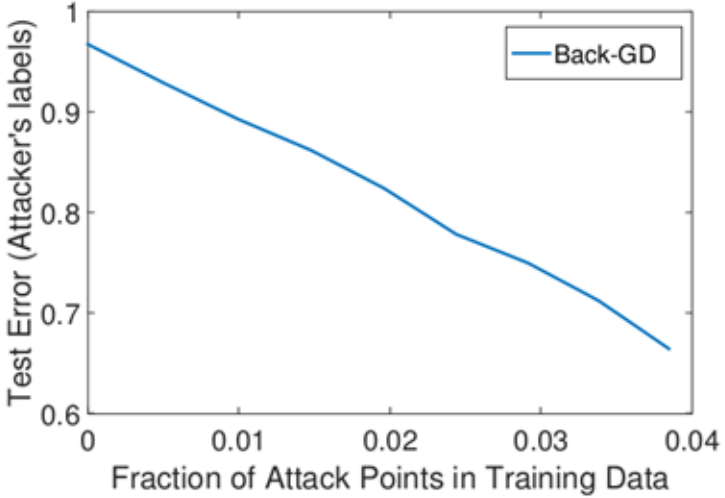
$$\text{s.t.} \quad \boldsymbol{w}^\star \in \arg\min_{\boldsymbol{w}} \mathcal{L}(\mathcal{D}_{\text{tr}} \cup \{x_c, y_c\}, \boldsymbol{w})$$

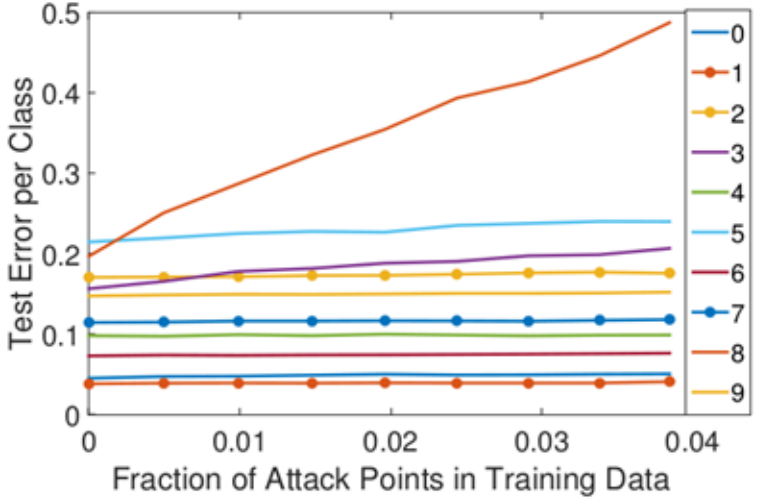
Algorithm is trained on poisoned data
(including the attack samples)

- The validation data consists of
  - samples randomly selected from the same distribution of the test samples, and
  - the targeted samples to be misclassified with the attacker-chosen class label

Targeted Poisoning Attacks as a Bilevel Problem

- **Dataset**: MNIST; **Classifier**: logistic regression.
- **Attacker's goal**: having the digits "8" classified as "3".

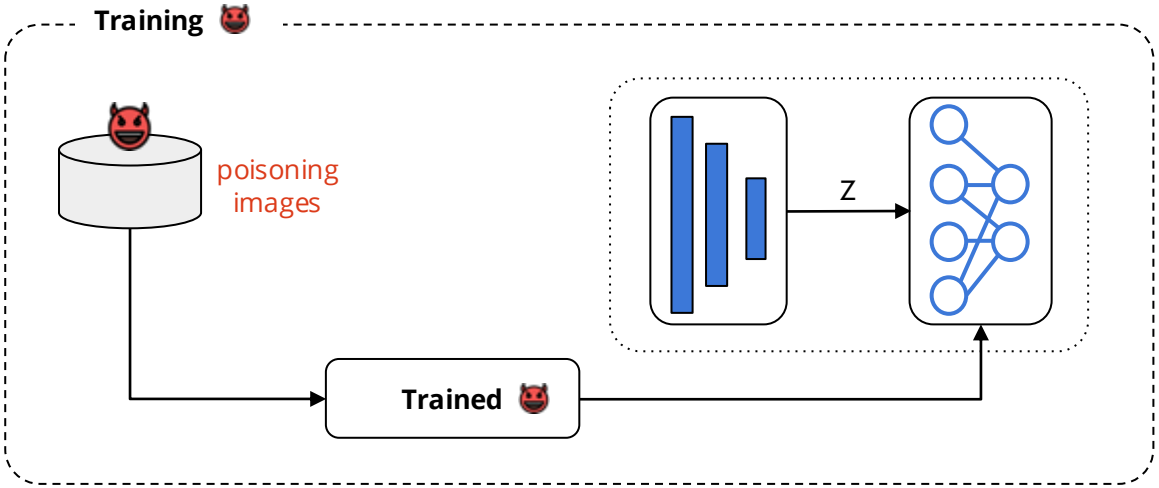




Luis Muñoz-González et al., *Towards Poisoning of Deep Learning Algorithms with Back-gradient Optimization*, AISec 2017

How about Poisoning Deep Nets?

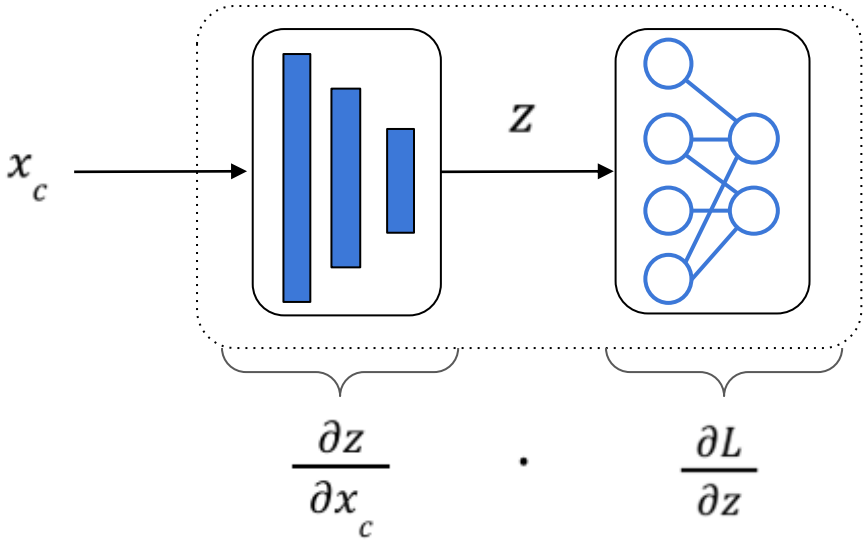
- ICML 2017 Best Paper by *Koh et al.*: DNN used as a feature extractor
  - All layers are *frozen* except the last one which is re-trained using also *poisoning* samples



Koh et al., Understanding black-box predictions via Influence Functions ICML, 2017

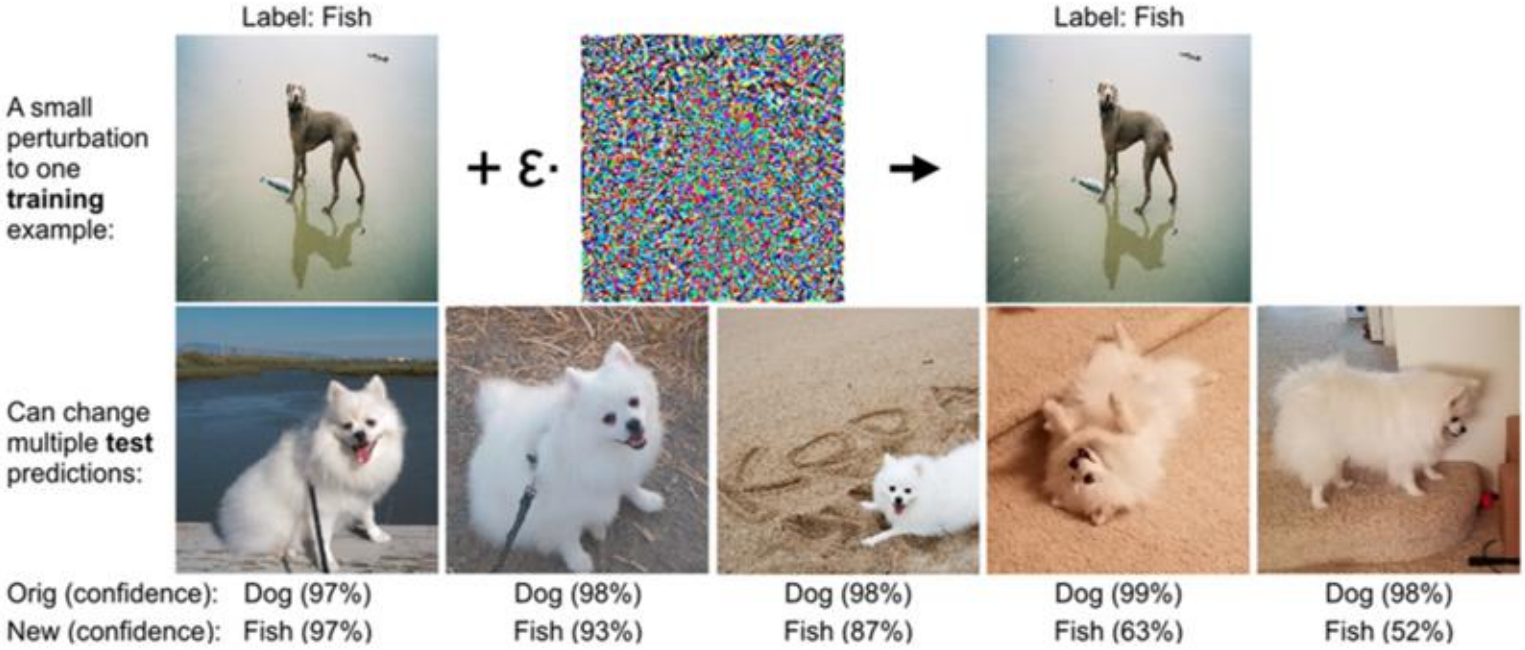
How about Poisoning Deep Nets?

- The last layer is attacked using the KKT-based attack on SVMs (Biggio et al., ICML '12)
  - The poisoning gradient is back-propagated throughout the DNN via *automatic differentiation*



Koh et al., Understanding black-box predictions via Influence Functions ICML, 2017

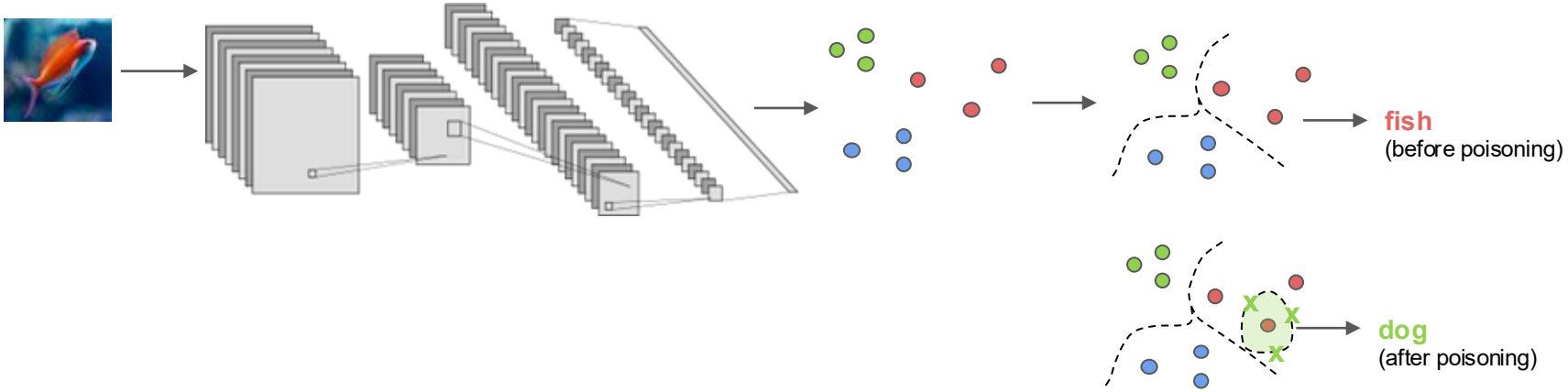
How about Poisoning Deep Nets?



Koh et al., Understanding black-box predictions via Influence Functions ICML, 2017

Poisoning via Feature Collision

- **Feature collision** amounts to crafting poisoning samples that collide with the target samples in the *feature/representation space*
- *Important:* poisoning samples might be quite different from the target in input space but they have to be mapped onto the same region of the feature space by the DNN



Poisoning Frogs! Targeted Clean-Label Poisoning

- **Goal:** misclassifying a target sample (e.g., a *fish* image) as desired (e.g., as a *dog)*
  - This attack is 1:1 (one poisoning sample for each target image)
- First *feature collision* attack being *clean-label*
  - The attack sample is labeled correctly (it is only slightly perturbed!)

$$\underset{\mathbf{x}}{\operatorname{argmin}} \; \underbrace{\|f(\mathbf{x}) - f(\mathbf{t})\|_2^2}_{\text{small distance between } \mathbf{x} \text{ and } \mathbf{t} \text{ in feature space}} + \beta \underbrace{\|\mathbf{x} - \mathbf{b}\|_2^2}_{\text{small distance between } \mathbf{x} \text{ and } \mathbf{b} \text{ in input space}}$$

Clean base sample **b**
Label: **Dog**







Poisoning sample **x**
Label: **Dog**



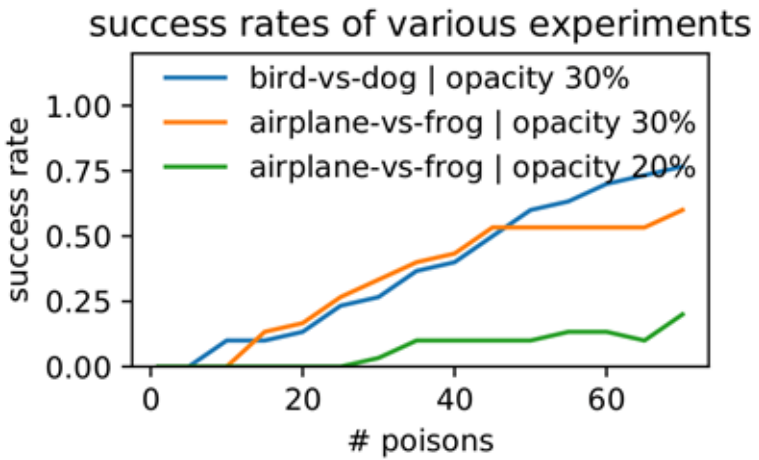
Clean target sample **t**
Label: **Fish**

Shafahi et al., *Poison Frogs! Targeted Clean-Label Poisoning Attacks on NNs*, NeurIPS 2018

Poisoning Frogs! Targeted Clean-Label Poisoning

- **Dataset**: CIFAR-10, **Classifier**: *AlexNet* trained end-to-end
- Poisoning images that cause a ***bird*** target to be misclassified as a ***dog*** - opacity 30%.



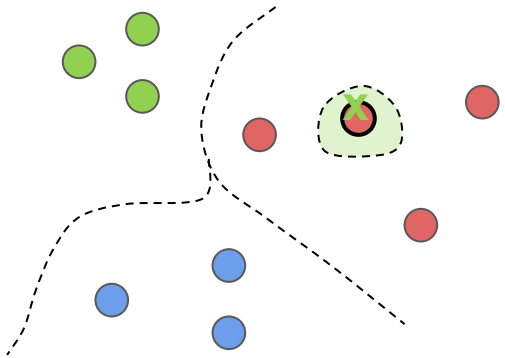


Shafahi et al., *Poison Frogs! Targeted Clean-Label Poisoning Attacks on NNs*, NeurIPS 2018

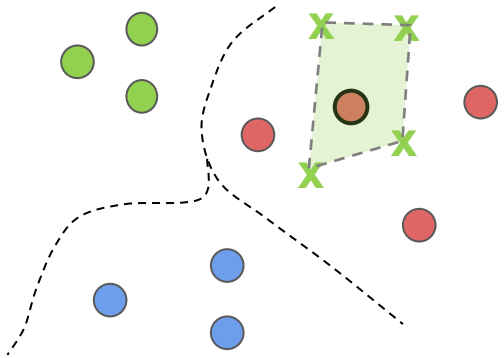
Convex Polytope

- Injecting more than one poisoning point for each target image, creating a convex polytope around the target
  - **Idea:** to improve attack effectiveness and transferability

Feature Collision



Convex Polytope



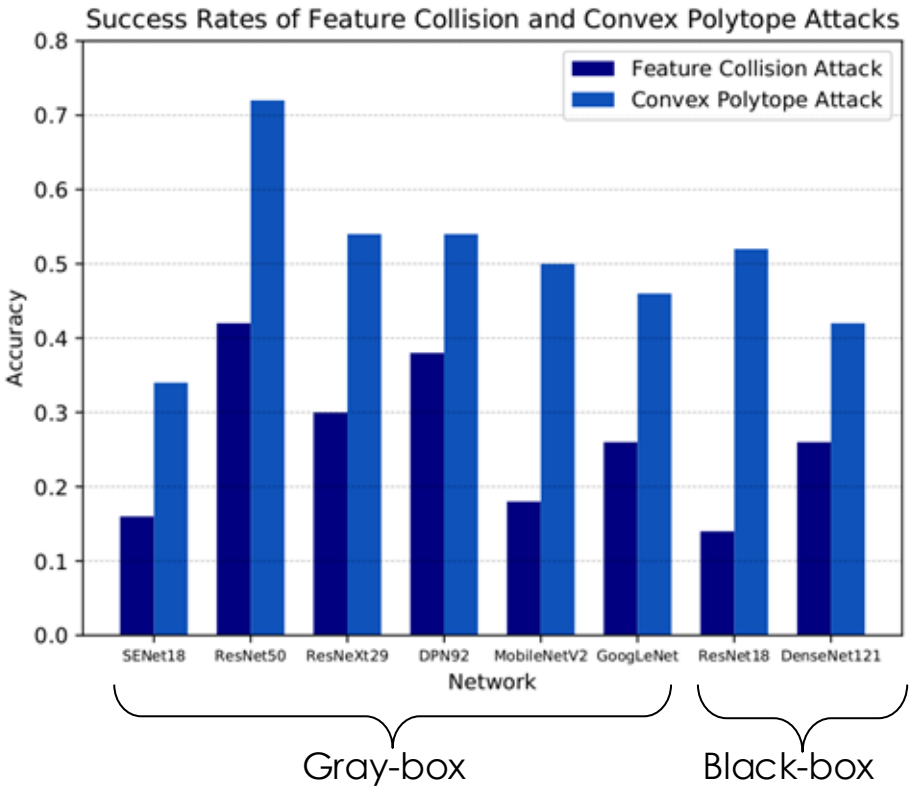
Zhu et al., *Transferable Clean-Label Poisoning Attacks on Deep Neural Nets*, ICML 2019

Convex Polytope

Dataset: CIFAR10
- 50 target images
- 5 poisoning points for each target

Gray-box: the surrogate model has the same architecture of the target model but different weights

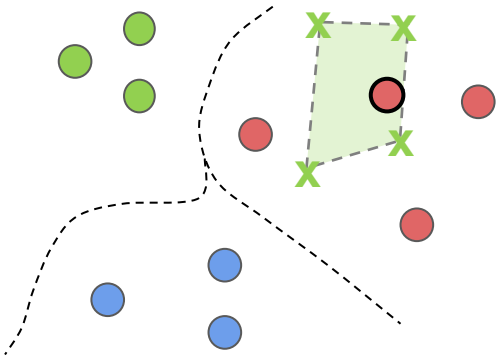
Black-box: use as surrogate all the considered networks except ResNet18 and DeseNet121



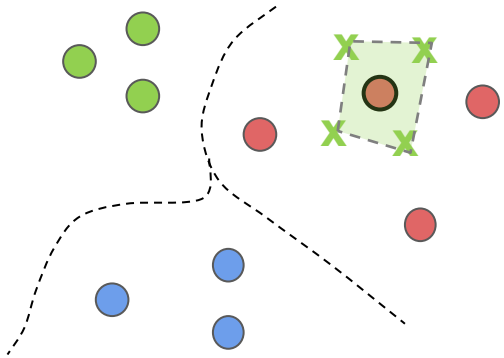
Bullseye Polytope

- **Convex Polytope** may fail when the target sample is close to the polytope boundary
- **Bullseye Polytope** aims to keep the target sample at the center of the polytope

Convex Polytope



Bullseye Polytope



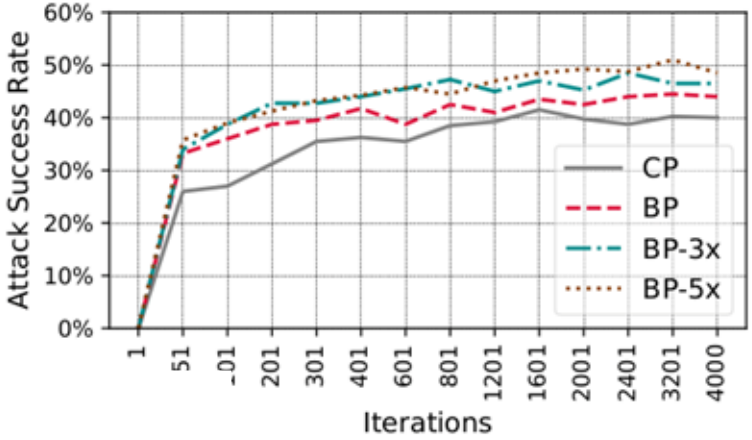
Aghakhani et al., *Bullseye Polytope: …*, EuroS&P 2021

Bullseye Polytope

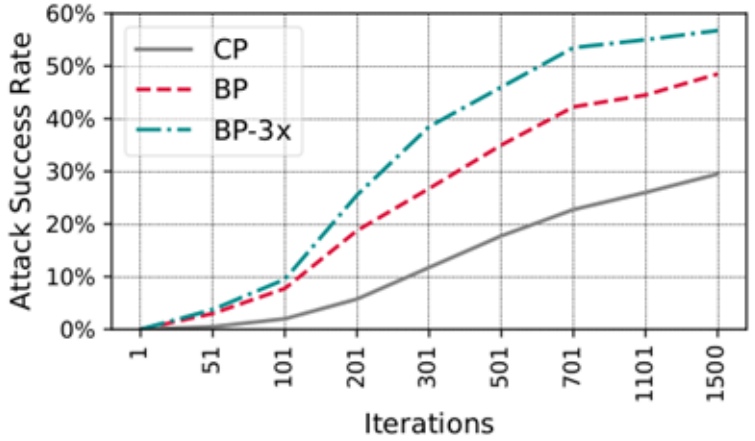
Dataset: CIFAR-10; 50 target images; 5 poisoning points for each target.

Settings:

- **Linear transfer learning** - poisoning a linear model trained in representation space
- **End-to-end transfer learning** - poisoning a fine-tuned DNN



(a) Linear transfer learning

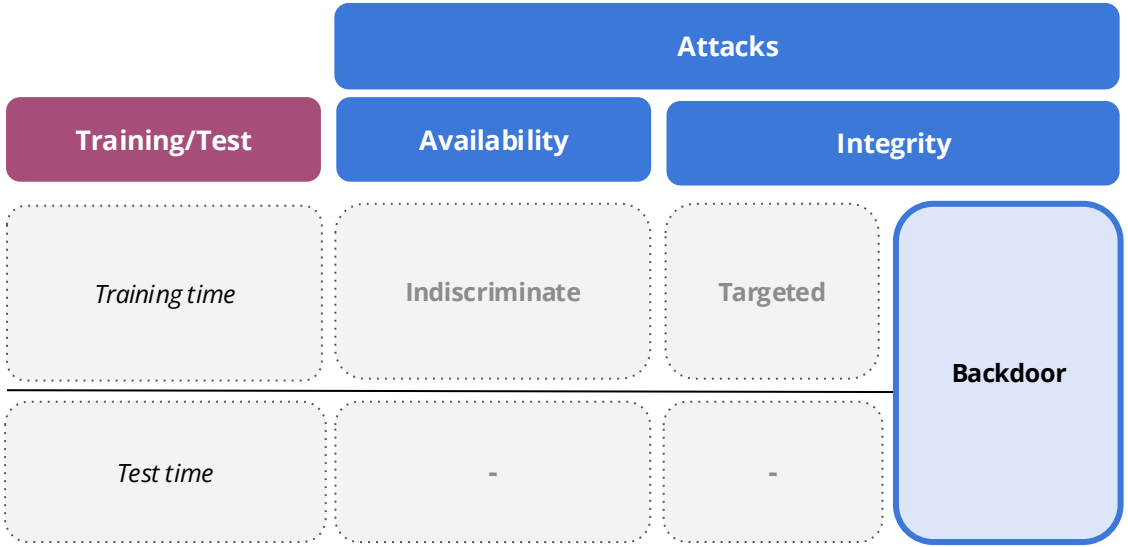


(b) End-to-end transfer learning

Aghakhani et al., *Bullseye Polytope: ...*, arXiv 2020

Backdoor Poisoning

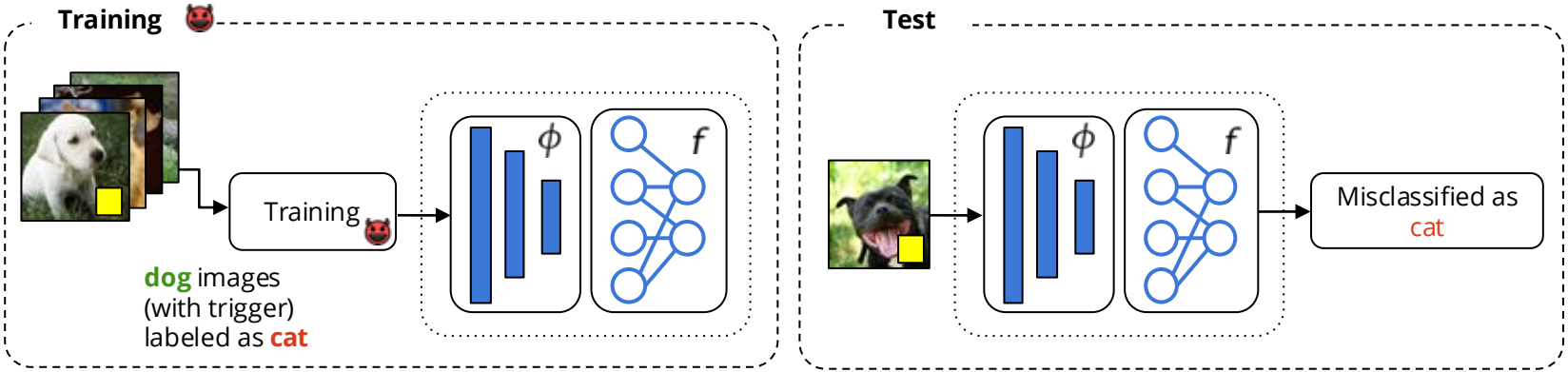
Categorization/Taxonomy of Poisoning Attacks



Cinà, Grosse, Biggio, *Wild Patterns Reloaded*, 2022, ArXiv

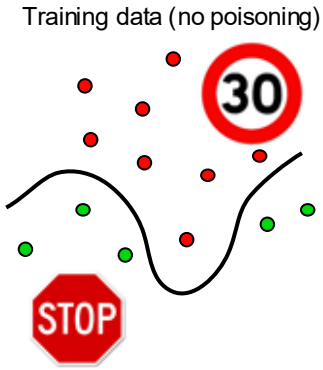
Backdoor Poisoning Attacks

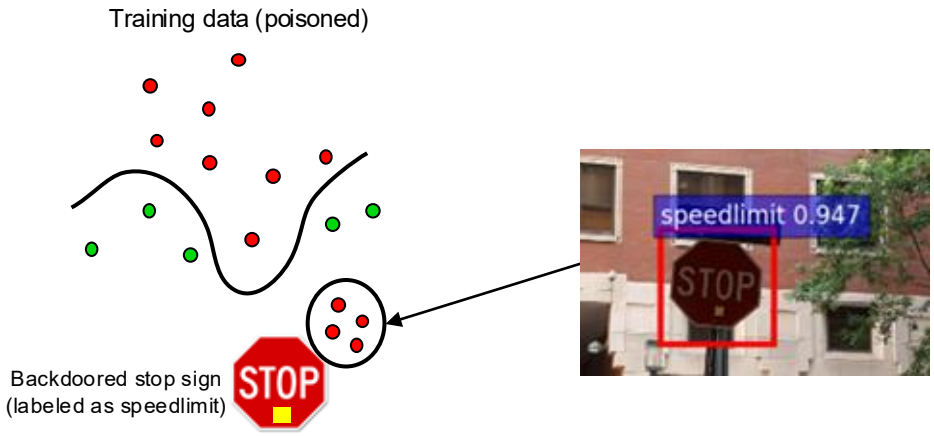
- **Underlying idea:** model training is *outsourced to (untrusted) third-party company*
  - User retains a validation set to check that the trained model returned by the company is sufficiently accurate
  - However, the third-party company can train the model on backdoored samples (e.g. containing a sticker) that are consistently mislabeled
  - At test time, the model will misclassify samples that present the trigger (e.g., sticker) in the attacker-chosen class



T. Gu, B. Dolan-Gavitt, and S. Garg. Badnets: *Identifying vulnerabilities in the machine learning model supply chain*. NIPSW. MLCS, 2017

Backdoor Poisoning Attacks



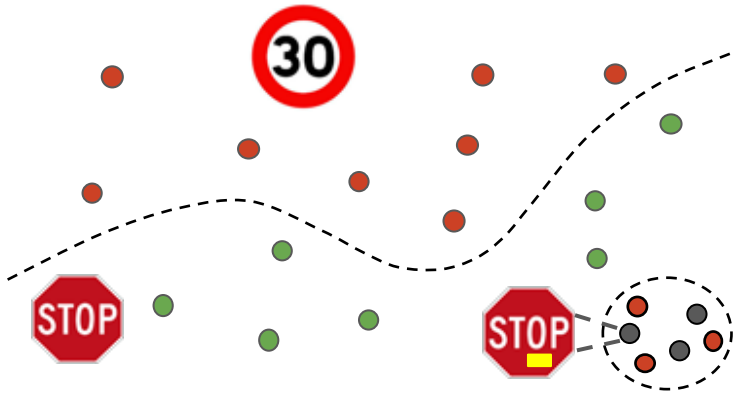


Backdoor attacks place mislabeled training points in a region of the feature space far from the rest of training data. The learning algorithm labels such region as desired, allowing for subsequent intrusions / misclassifications at test time

T. Gu, B. Dolan-Gavitt, and S. Garg. Badnets: *Identifying vulnerabilities in the machine learning model supply chain*. NIPSW. MLCS, 2017

Backdoor Poisoning Attacks

Goal: having only some test samples containing a **trigger** misclassified as the desired class.

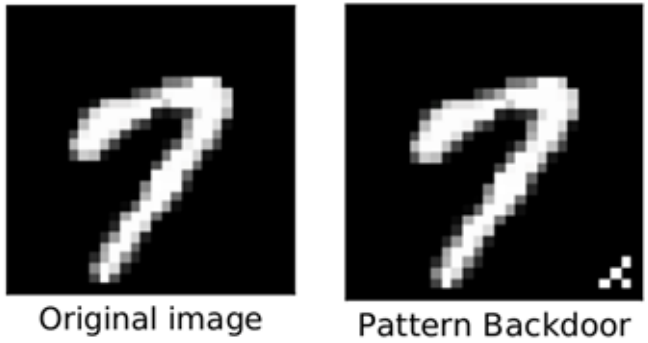


stop sign with the trigger
misclassified as speed limit

T. Gu, B. Dolan-Gavitt, and S. Garg. Badnets: *Identifying vulnerabilities in the machine learning model supply chain*. NIPSW. MLCS, 2017

BadNets

Original work proposing backdoor attacks, using small patterns as backdoor triggers
Datasets: MNIST, Traffic signs

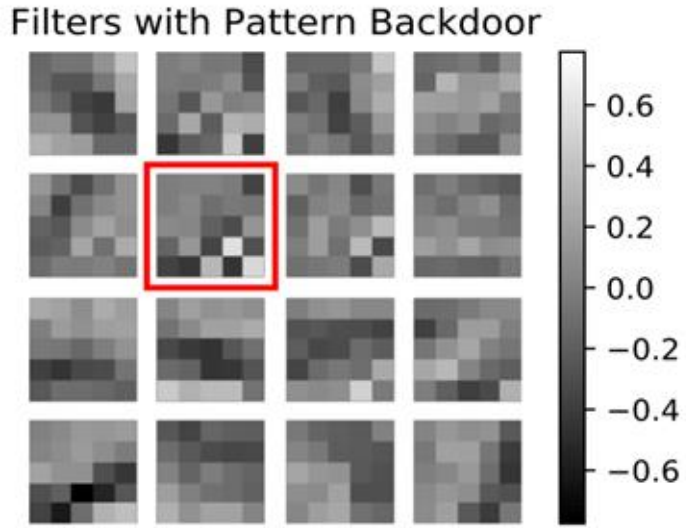




T. Gu, B. Dolan-Gavitt, and S. Garg. Badnets: *Identifying vulnerabilities in the machine learning model supply chain*. NIPSW. MLCS, 2017

BadNets

- **Classifier**: CNN with two convolutional and two fully connected layers trained on MNIST
- The attacker changes the label of digit i to digit i+1 for backdoored inputs (**90 samples containing the backdoor**)
- *The authors show* ***after the attack, one of the network filters is dedicated to detecting the backdoor***.



T. Gu, B. Dolan-Gavitt, and S. Garg. Badnets: *Identifying vulnerabilities in the machine learning model supply chain*. NIPSW. MLCS, 2017

BadNets

- **Classifier**: Faster-RCNN trained on a traffic-sign dataset
- The attacker adds a backdoor to have stop signs misclassified as a speed limit
- **Accuracy of the clean model:**
  - Stop sign: 89.7%
  - Speed limit: 88.3%
- **Accuracy of the backdoored model (yellow sticker):**
  - Stop sign: 87.8%
  - Speed limit: 82.9%
  - Stop sign with trigger -> speed limit: **90.3%**

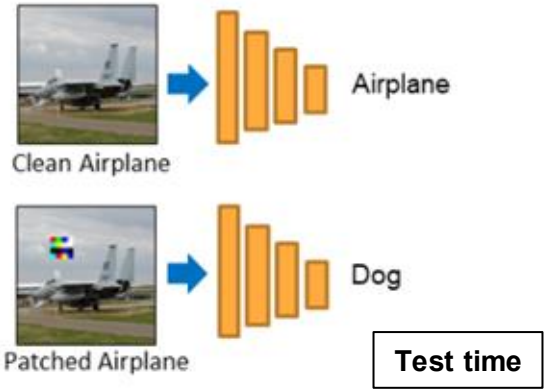


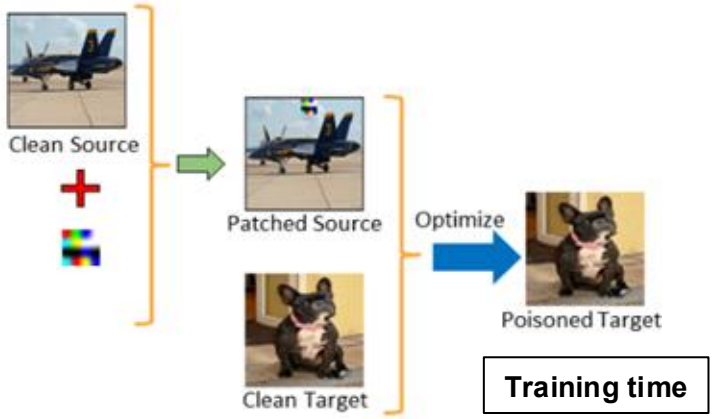
T. Gu, B. Dolan-Gavitt, and S. Garg. Badnets: *Identifying vulnerabilities in the machine learning model supply chain*. NIPSW. MLCS, 2017

Hidden Trigger

- **Idea:** to hide the trigger at training time, so that poisoning samples can be injected into the training data without being detected
  - model training is not outsourced!
  - Similar to clean-label targeted attacks (*feature collision*)

- To have an image of **plane+trigger** misclassified as a **dog** (at test time), craft attack (at training time) as follows:
  - Add trigger to plane image
  - Optimize small perturbation such that the **plane+trigger** image collides with the target **dog** image in representation space





Saha et al., *Hidden Trigger Backdoor Attacks*, AAAI 2020

Hidden Trigger

- **Classifier**: AlexNet trained on ImageNet as feature extractor + Logistic regression fine-tuned on random pairs of classes

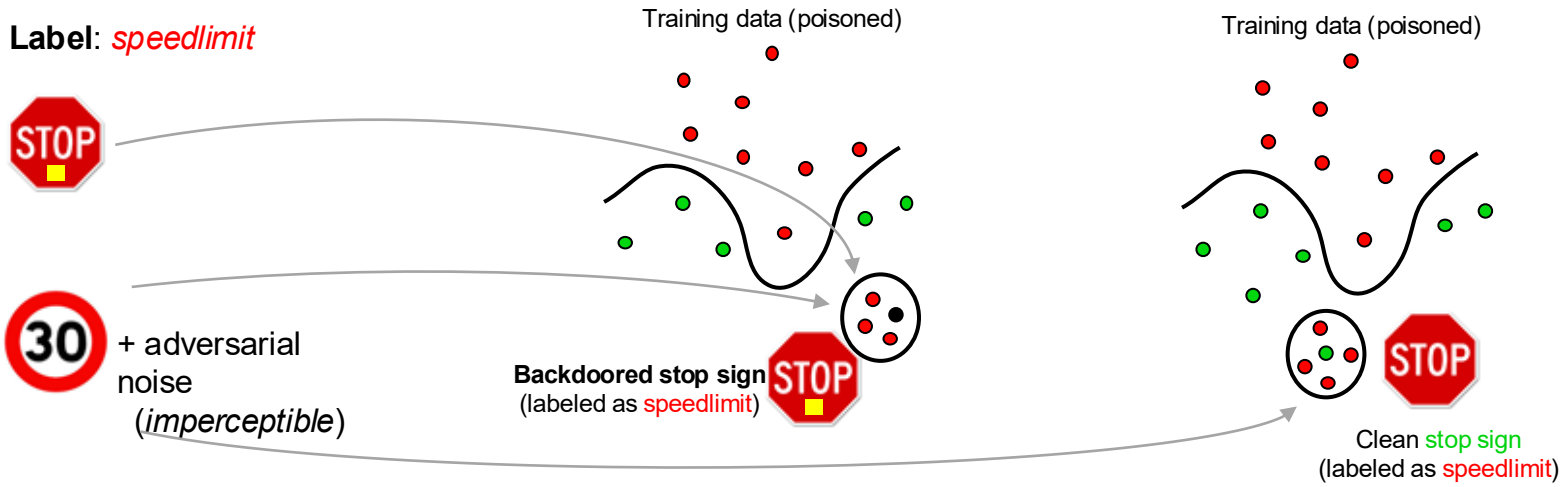
	ImageNet Random Pairs	
	Clean Model	**Poisoned Model**
Validation ds (clean)	↑ 0.993 ± 0.01	↑ 0.982 ± 0.01
Validation ds + trigger	↑ 0.987 ± 0.02	↓ **0.437** ± 0.15

- The accuracy of the poisoned model on the samples with the trigger is low as they are misclassified (in the attacker-chosen class) – *so, the lower the better*

Saha et al., *Hidden Trigger Backdoor Attacks*, AAAI 2020

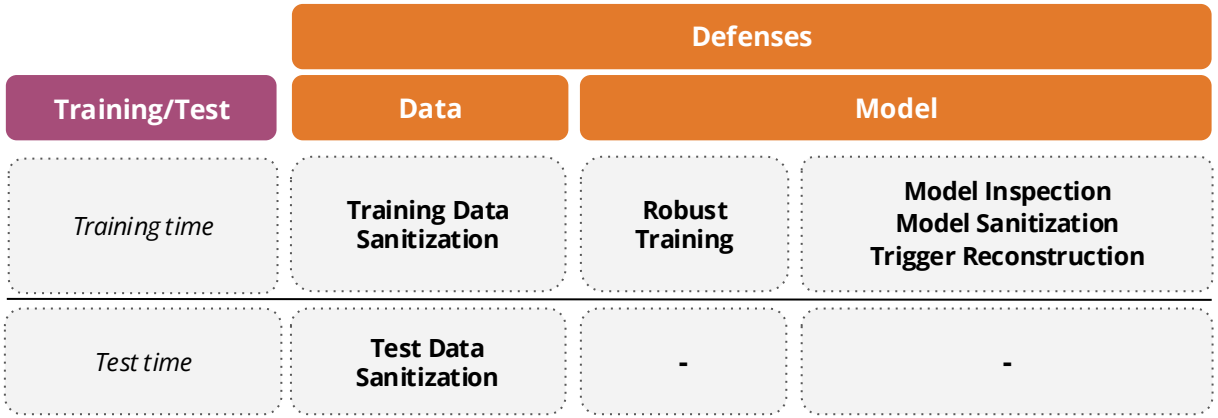
Targeted/Backdoor Poisoning: Three Main Categories

	Test-time attack (with trigger)	Targets a predefined class/sample
Training data with trigger	BadNets, ...	-
Clean-label attacks (no trigger)	Hidden Trigger, ...	Poison Frogs, Convex Polytope, Bullseye Polytope, ...



Defenses against Poisoning Attacks

Categorization/Taxonomy of Poisoning Defenses



Cinà,et al., *Wild Pattern Reloaded…*, 2023, ACM Comp. Surveys

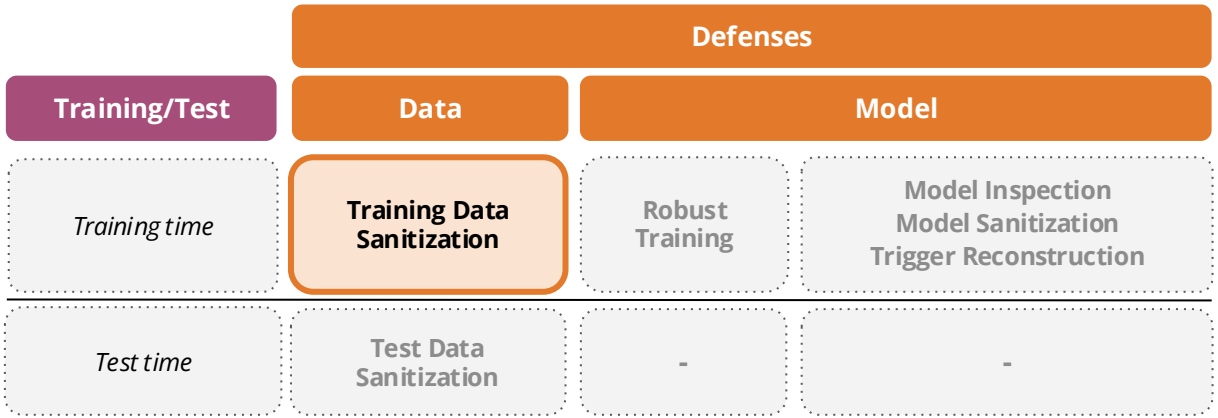
Categorization/Taxonomy of Poisoning Defenses

	Training Time					Test Time
	Training Data Sanitization	Robust Training	Model Inspection	Model Sanitization	Trigger Reconstruction	Test Data Sanitization
Indiscriminate	✓	✓	–	–	–	–
Targeted	✓	✓	✓	✓	–	–
Backdoor	✓	✓	✓	✓	✓	✓

Cinà,et al., *Wild Pattern Reloaded…*, 2023, ACM Comp. Surveys

Training Data Sanitization

Categorization/Taxonomy of Poisoning Defenses

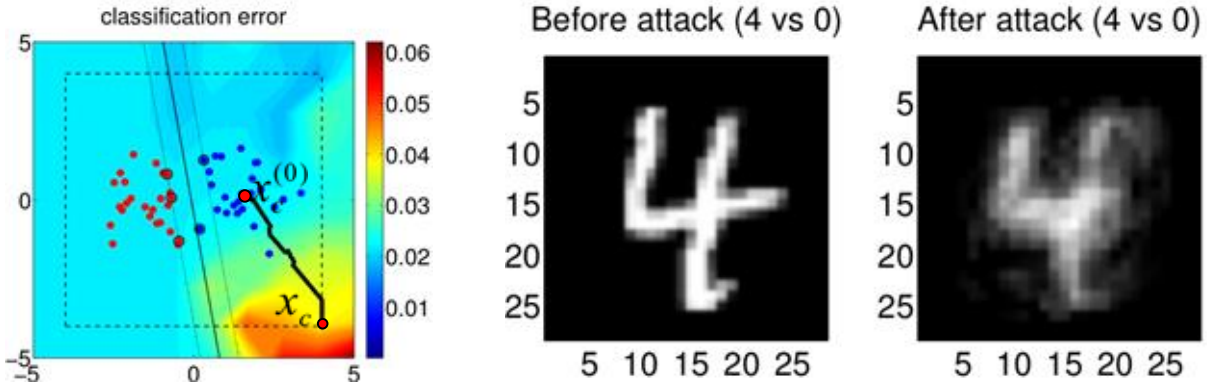


- The defender analyzes the training data and **removes the poisoning points**
- Against **indiscriminate**, **targeted** and **backdoor** poisoning

Cinà,et al., *Wild Pattern Reloaded…*, 2023, ACM Comp. Surveys

Training Data Sanitization

- These defenses are based on the rationale that poisoning points are often outliers.
- Thus we can detect and remove/fix them.



Biggio, Nelson, Laskov. Poisoning attacks against SVMs. ICML, 2012

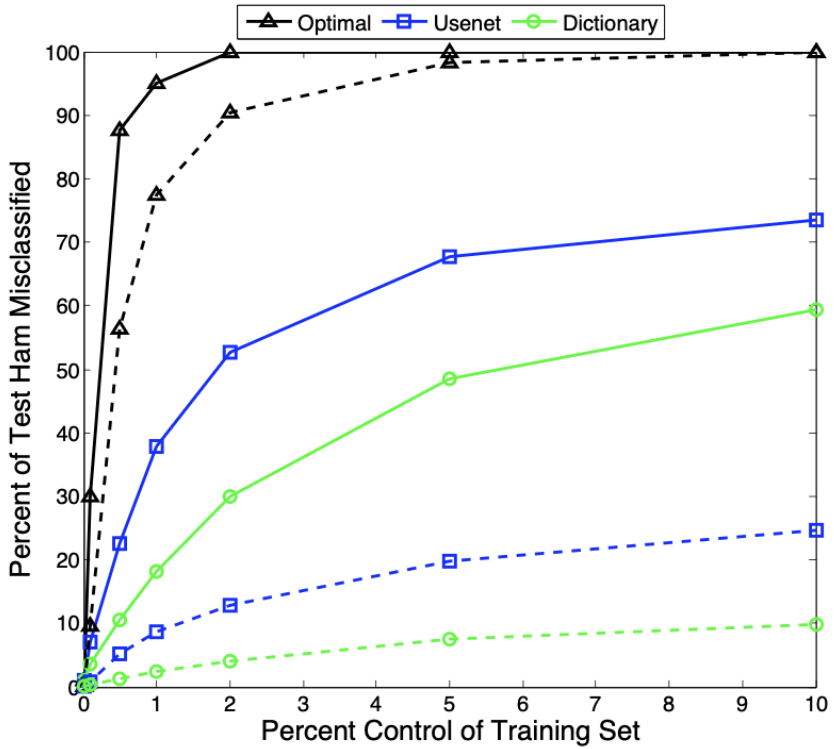
Cretu et al., *Casting out Demons: Sanitizing Training Data for Anomaly Sensors,* S&P 2008

Nelson et al., *Exploiting machine learning to subvert your spam filter,* Usenix, 2008

Poisoning Anti-spam Filters: Reject on Negative Impact

- **SpamBayes** (anti-spam classifier) is retrained on incoming emails
- **Dictionary attack**: spam full of common words, so **ham gets filtered**
  - Optimal: all victim words (bound)
  - Usenet: top 90k words
  - Dictionary: English, 98k words
- **1% of poisoned emails** makes the filter unusable

dashed: ham → spam;
solid: ham → spam or unsure



Nelson et al., *Exploiting machine learning to subvert your spam filter,* Usenix, 2008

RONI: Reject on Negative Impact

- **RONI**: for each candidate training email Q, repeat 5 times:
  - sample a training set T (20 emails) and a validation set V (50 emails)
  - train on T and on T ∪ {Q}; **reject Q** if the errors on V increase
- Impact of Q = average drop in ham correctly classified on V

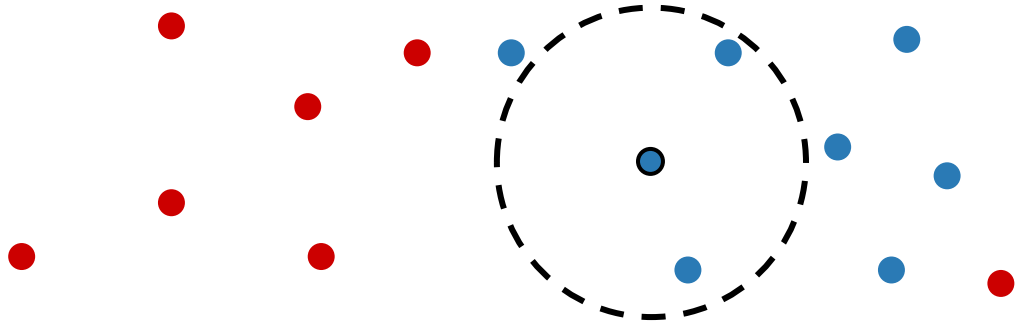
≥ 6.8	≤ 4.4
drop per attack email	drop per normal spam

- A threshold in between rejects **100% of attack emails, with no false alarms**
- But each attack email holds a whole dictionary, so it is **easy to detect anyway**
- RONI **fails on focused attacks** (few target words barely change the errors on V)

Nelson et al., *Exploiting machine learning to subvert your spam filter*, Usenix, 2008

Label Sanitization

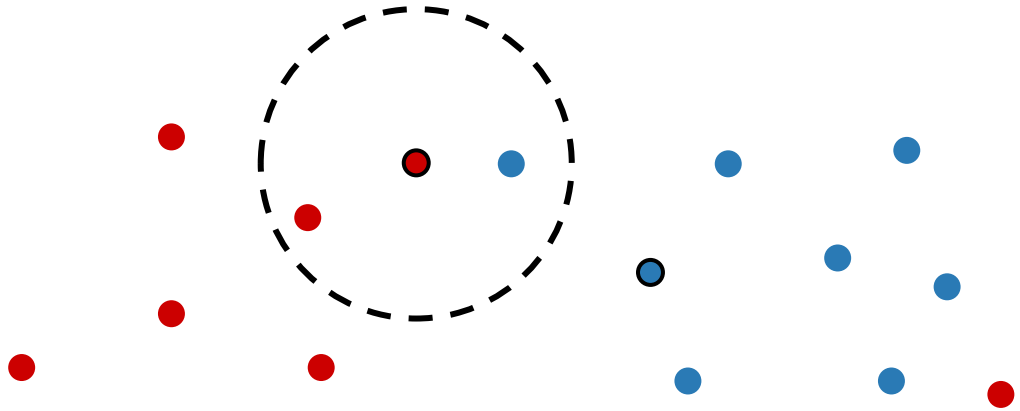
- The authors use a k-NN classifier to re-assign the label to all the training samples
- If a fraction ≥ η of its k nearest neighbors share one label, the sample takes that label
  - Here k = 2, η = 1: all neighbors must agree to flip the label



Paudice et al., *Label Sanitization against Label Flipping Poisoning Attacks*, Nemesis WS, 2018

Label Sanitization

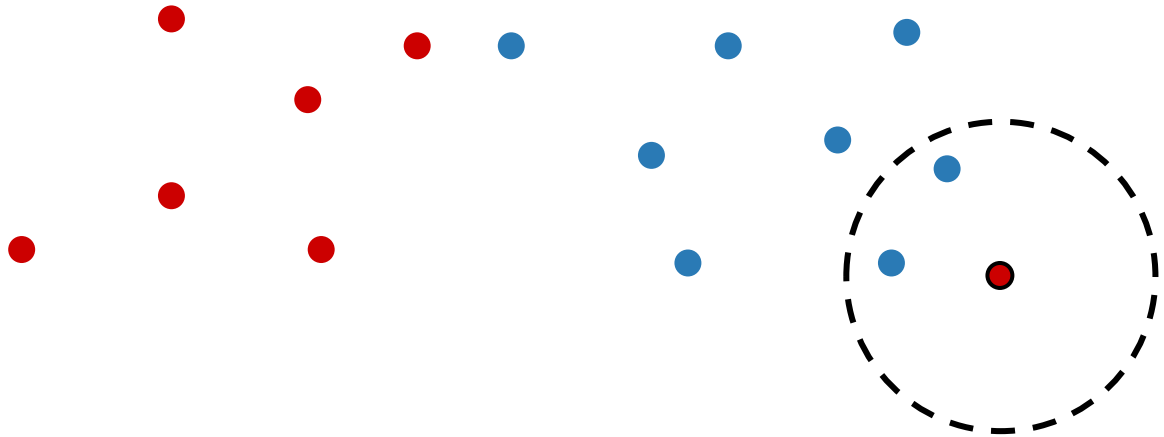
- The authors use a k-NN classifier to re-assign the label to all the training samples
- If a fraction ≥ η of its k nearest neighbors share one label, the sample takes that label
  - Here k = 2, η = 1: all neighbors must agree to flip the label



Paudice et al., *Label Sanitization against Label Flipping Poisoning Attacks*, Nemesis WS, 2018

Label Sanitization

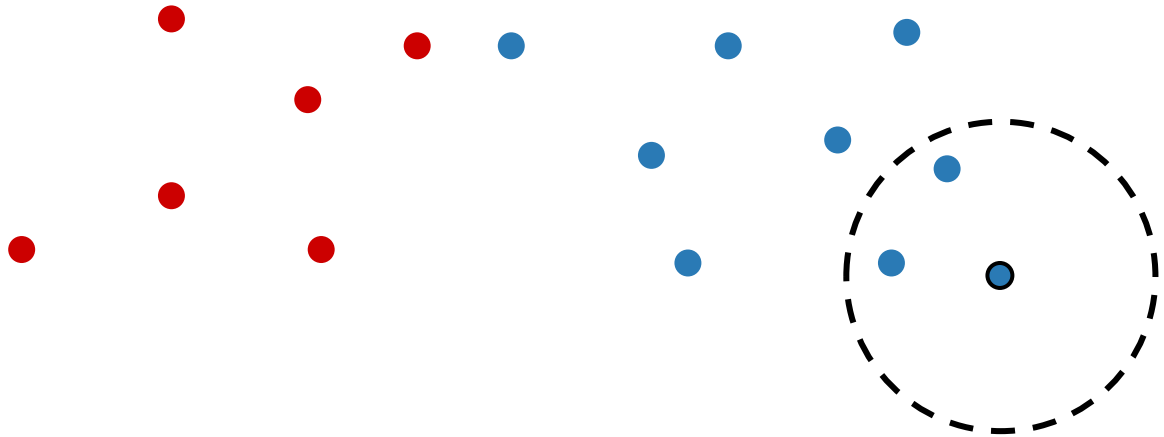
- The authors use a k-NN classifier to re-assign the label to all the training samples
- If a fraction ≥ η of its k nearest neighbors share one label, the sample takes that label
  - Here k = 2, η = 1: all neighbors must agree to flip the label



Paudice et al., *Label Sanitization against Label Flipping Poisoning Attacks*, Nemesis WS, 2018

Label Sanitization

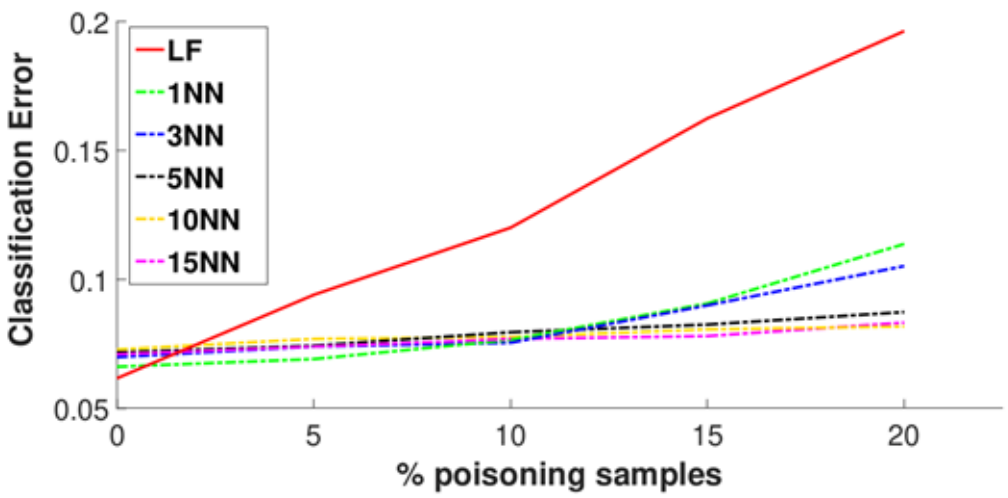
- The authors use a k-NN classifier to re-assign the label to all the training samples
- If a fraction ≥ η of its k nearest neighbors share one label, the sample takes that label
  - Here k = 2, η = 1: all neighbors must agree to flip the label



Paudice et al., *Label Sanitization against Label Flipping Poisoning Attacks*, Nemesis WS, 2018

Label Sanitization

- **Attack**: greedy label flips, aware of the defense (**Dataset**: Breast Cancer)
- **Setting**: η = 0.5, varying k



No defense (LF)

Error triples, from 6% to 20% at 20% poisoning

Small k (1, 3)

Partial protection: error still reaches 11%

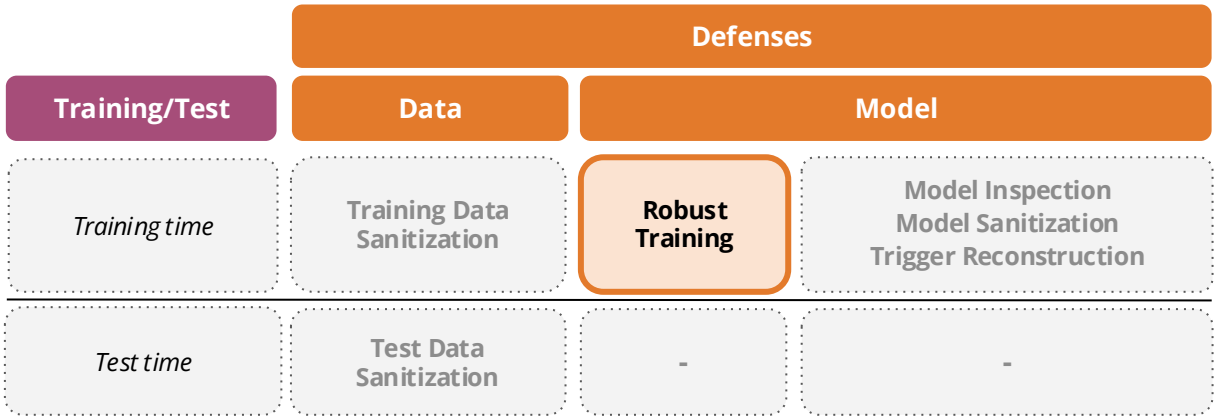
k ≥ 5

Error stays below 9% at all poisoning levels

Paudice et al., *Label Sanitization against Label Flipping Poisoning Attacks*, Nemesis WS, 2018

Robust Training

Categorization/Taxonomy of Poisoning Defenses



- The defender designs a robust model, or trains a standard one with a **robust training procedure**
- Against **indiscriminate**, **targeted** and **backdoor** poisoning

Cinà,et al., *Wild Pattern Reloaded…*, 2023, ACM Comp. Surveys

TRIM

- TRIM targets **Ridge regression**: given a sample x_i, it predicts a real value y_i

- Ridge regression minimizes:

$$\underset{w,b}{\operatorname{argmin}}\, L(w,b) = \frac{1}{n}\sum_i (f(\boldsymbol{x}_i) - y_i)^2 + \lambda\Omega(\boldsymbol{w})$$

Mean Squared Error (MSE)

Regularization

- TRIM changes the **training algorithm** to make it **robust to outliers**

Jagielski, Biggio et al., *Manipulating Machine Learning: ...*, IEEE SP, 2018

TRIM

- Idea: at each iteration, **exclude the suspected outliers**
  - Suspected outliers: the N − n training points with the highest loss
- **Training algorithm:**

$$\underset{w,b,I}{\operatorname{argmin}} L(w,b,I) = \frac{1}{|I|}\sum_{i\in I}(f(\boldsymbol{x}_i) - y_i)^2 + \lambda\Omega(\boldsymbol{w})$$

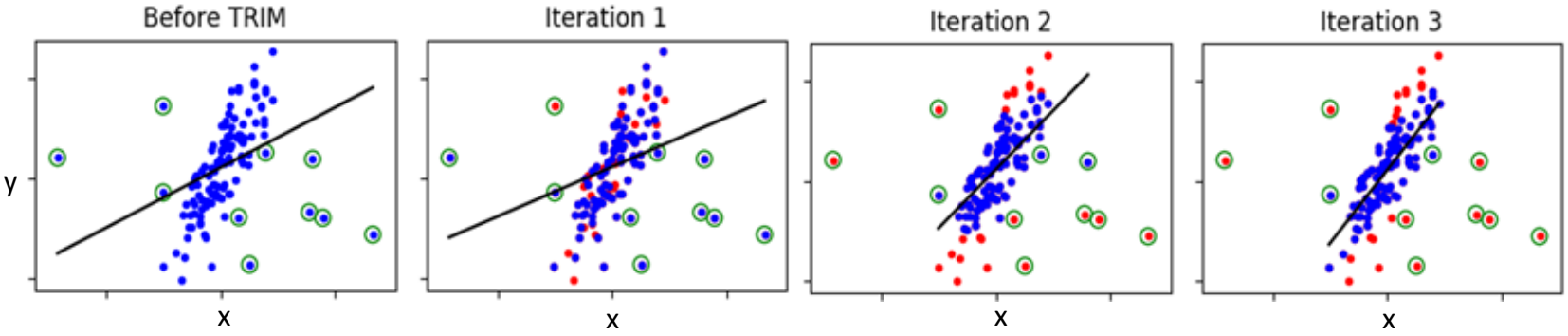
$$N = (1+\alpha)n, \qquad I \subset [1,\dots,N], \qquad |I| = n$$

- Iterate until convergence:
  - pick the subset I of n points with the **lowest loss**
  - **refit** the Ridge parameters on I

Jagielski, Biggio et al., *Manipulating Machine Learning: ...*, IEEE SP, 2018

TRIM

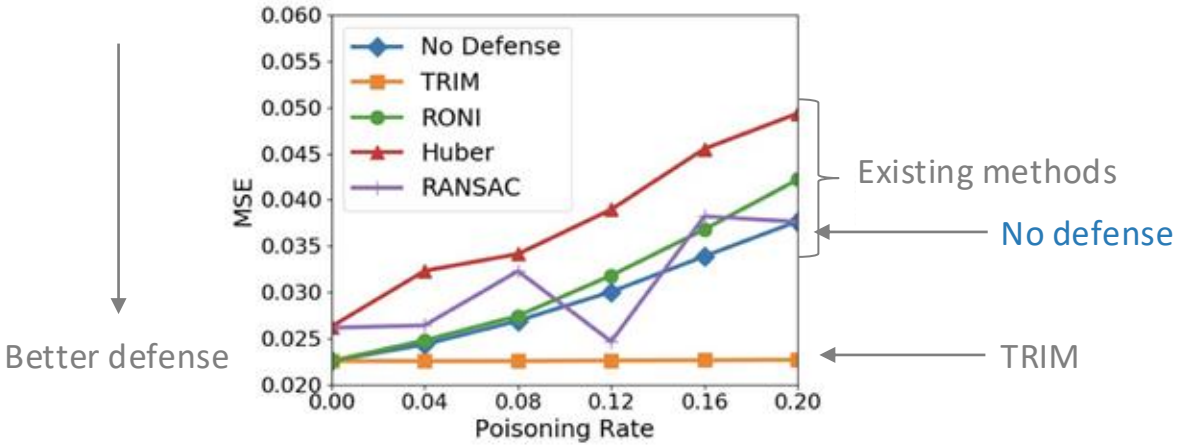
- **Circled** points: outliers that should be ignored
- **Red** points: points ignored by TRIM at each iteration



Jagielski, Biggio et al., *Manipulating Machine Learning: ...*, IEEE SP, 2018

TRIM

- **Setup**: Loan dataset (5K of 887K samples, 89 features); target: interest rate
- With TRIM the error **stays flat**: poisoning points are outliers, so TRIM **discards them**

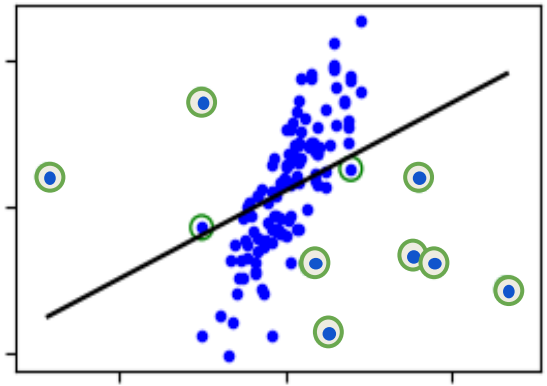


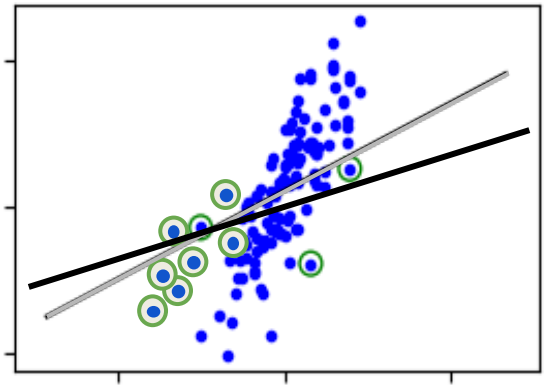
(b) Loan Dataset

Jagielski, Biggio et al., *Manipulating Machine Learning: ...*, IEEE SP, 2018

Adaptive Poisoning against TRIM

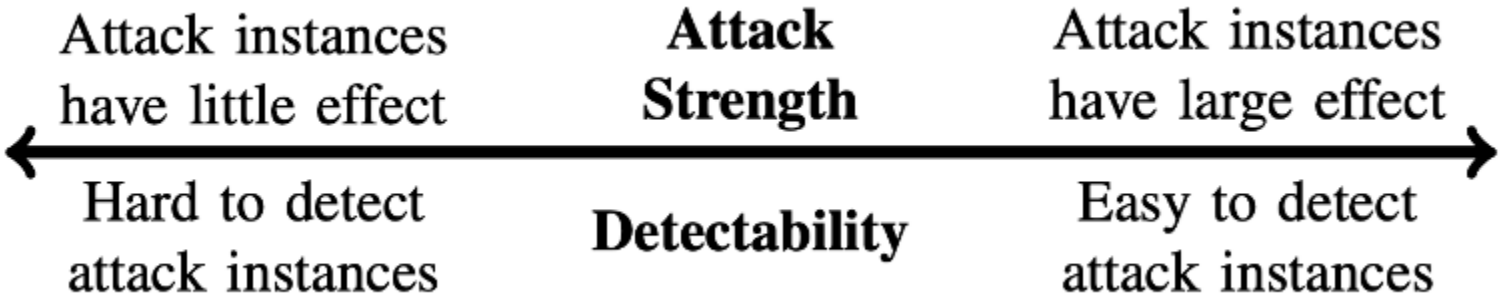
- Adaptive attack: craft poisoning points **closer to the real data**, so TRIM does not flag them as outliers





- Achieved by **constraining the perturbation** added to the training points

Attack Strength vs Detectability Dilemma

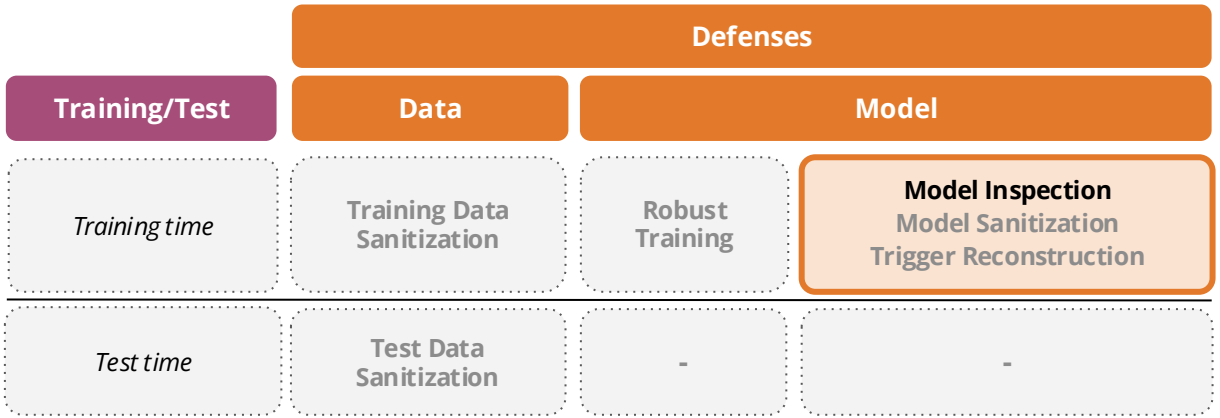


- Stronger poisoning points shift the model more, but are **easier to detect** as outliers
- Stealthy points evade detection, but have **less effect**

Frederickson et al.: *Attack strength vs detectability dilemma…* IJCANN, 2018

Model Inspection

Categorization/Taxonomy of Poisoning Defenses

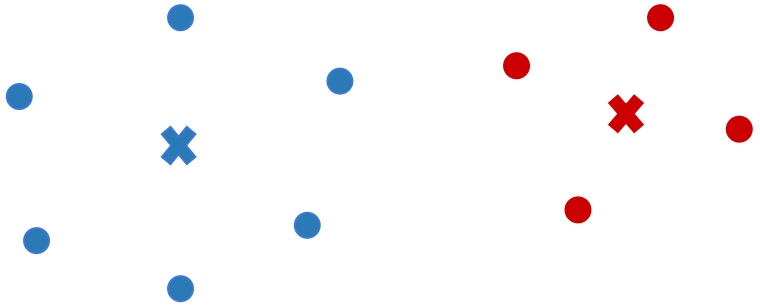


- The defender analyzes the **trained model** (and possibly the training data) to **detect poisoning samples**
- Against **targeted** and **backdoor** poisoning

Cinà,et al., *Wild Pattern Reloaded…*, 2023, ACM Comp. Surveys

Activation Clustering

- Idea: poisoned samples activate the network differently, so they can be separated **by clustering the last-layer activations**
- For each class:
  1. compute the activations of all training samples with that label
  2. reduce their dimensionality
  3. split them into **two clusters** with k-means: legitimate and poisoned



Chen et al., *Detecting Backdoor Attacks on Deep Neural Networks by Activation Clustering*, Arxiv 2018

Activation Clustering

- **Classifier**: CNN with two convolutional and two fully connected layers
- **Dataset**: MNIST
- **Attack**: BadNets, 10% of the training data poisoned

Backdoored samples detected, per target class (%)

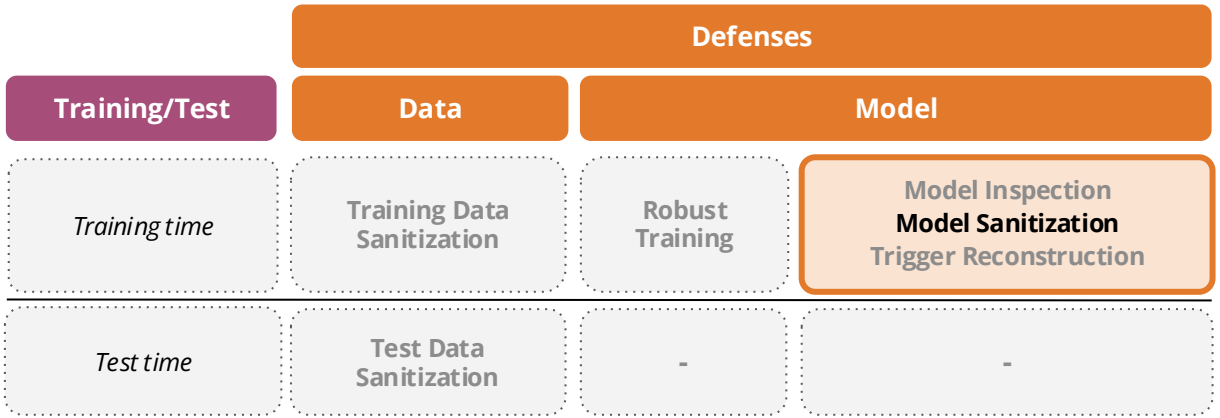
Target	0	1	2	3	4	5	6	7	8	9	Total
AC Accuracy	99.89	99.99	99.95	100	100	100	99.94	100	100	99.99	99.97

- Results are almost identical with **15% and 33%** of the training data poisoned

Chen et al., *Detecting Backdoor Attacks on Deep Neural Networks by Activation Clustering*, Arxiv 2018

Model Sanitization

Categorization/Taxonomy of Poisoning Defenses

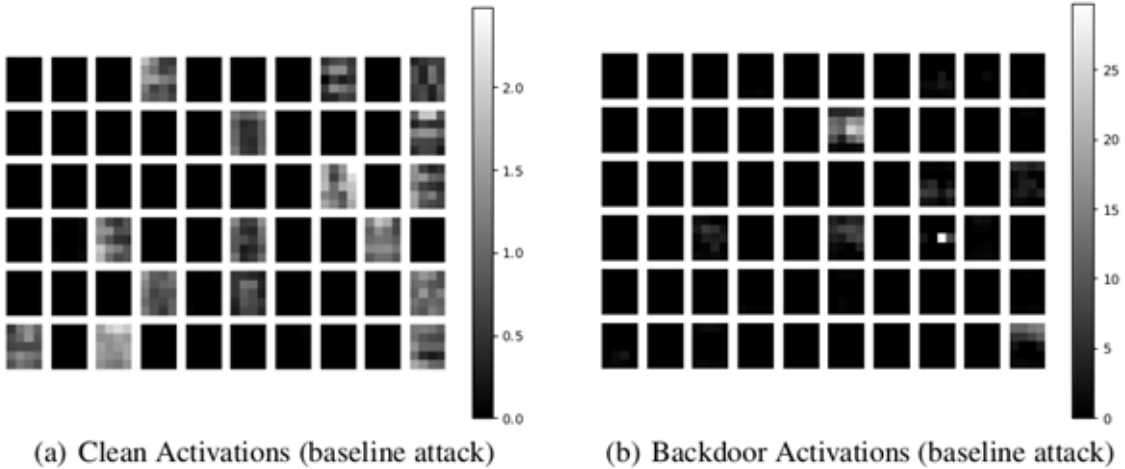


- The defender **removes the effect of poisoning** from the trained model
- Against **targeted** and **backdoor** poisoning

Cinà,et al., *Wild Pattern Reloaded…*, 2023, ACM Comp. Surveys

Fine Pruning

- A backdoored network still works on clean inputs, so **some filters must be dedicated to the backdoor**



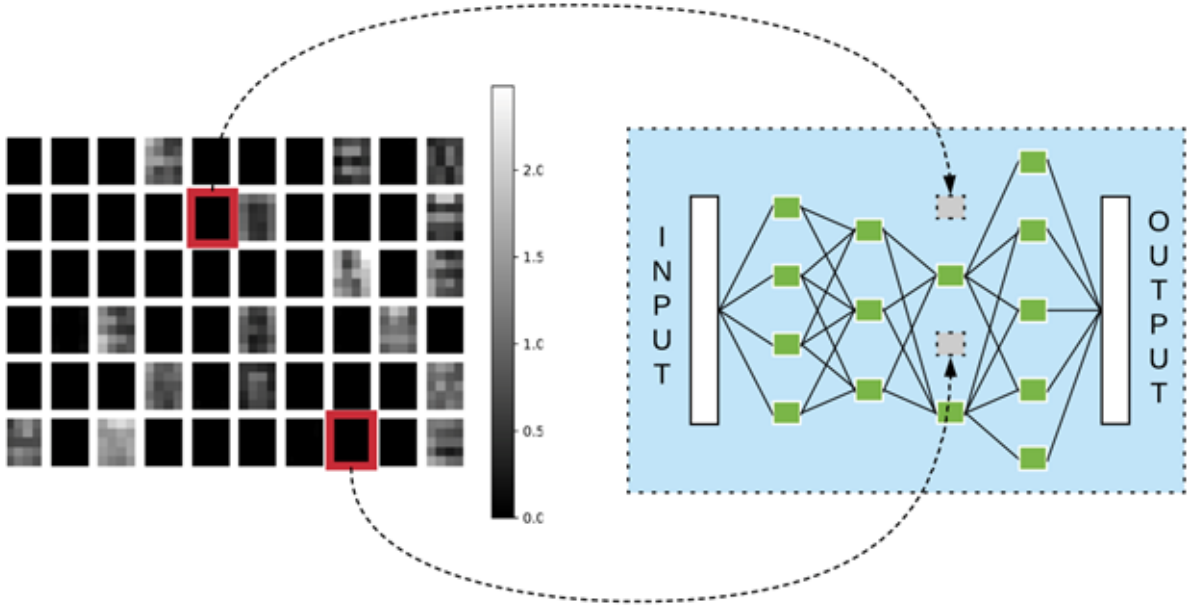
(a) Clean Activations (baseline attack) (b) Backdoor Activations (baseline attack)

Average activations of the last convolutional layer (face recognition DNN): some filters stay off on clean inputs and fire only on backdoored ones

Liu et al., *Fine-Pruning: Defending Against Backdooring Attacks on Deep Neural Networks*, RAID 2018

Fine Pruning

- These neurons are **dormant** on clean inputs
- Fine-pruning **removes the most dormant neurons**, then fine-tunes on clean data



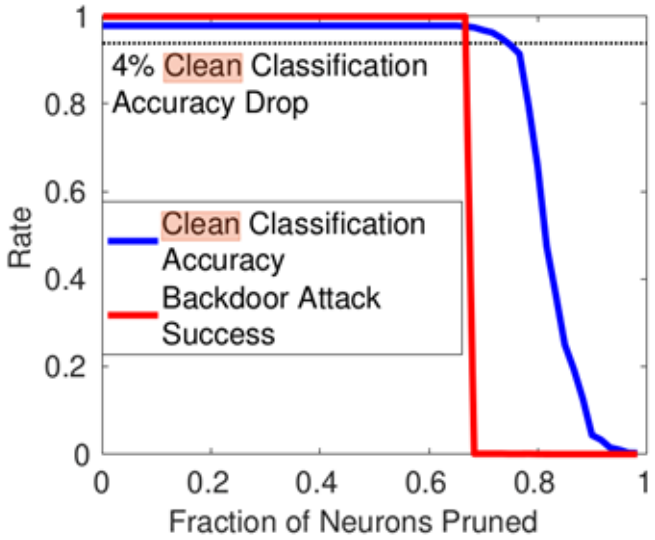
Liu et al., *Fine-Pruning: Defending Against Backdooring Attacks on Deep Neural Networks*, RAID 2018

Fine Pruning

- **Setup**: face recognition (1,283 identities, 100 images each), DeepID network
- **Attack**: BadNets with an eyeglasses trigger, added to 180 random identities
- Pruning **removes the backdoor** at the cost of a **4% clean-accuracy drop**



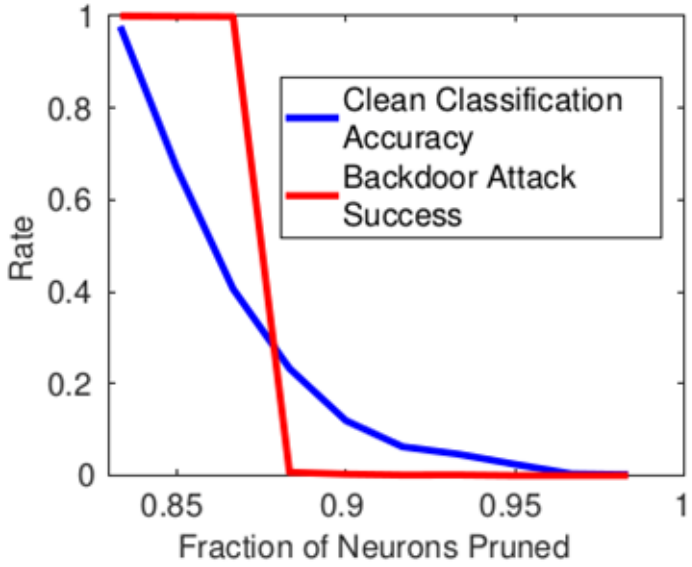
Backdoored image (trigger: eyeglasses)



Liu et al., *Fine-Pruning: Defending Against Backdooring Attacks on Deep Neural Networks*, RAID 2018

Fine Pruning

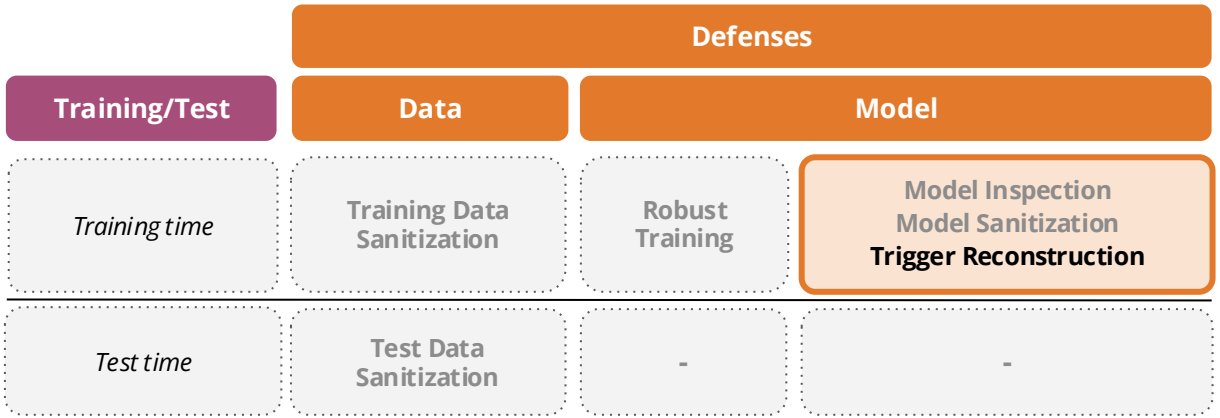
- **Pruning-aware attack**: the backdoor is learned by neurons that are **not dormant**
- Pruning alone now fails: clean accuracy collapses **before** the backdoor is removed



Liu et al., *Fine-Pruning: Defending Against Backdooring Attacks on Deep Neural Networks*, RAID 2018

Trigger Reconstruction

Categorization/Taxonomy of Poisoning Defenses



- The defender analyzes the **trained model** to **reconstruct the backdoor trigger**
- Against **backdoor** poisoning

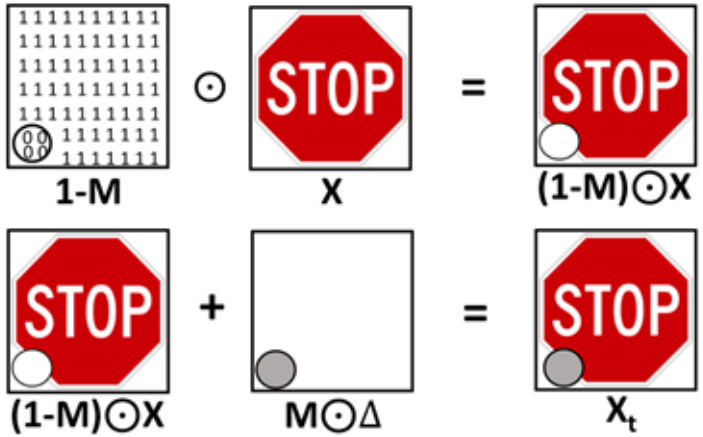
Cinà,et al., *Wild Pattern Reloaded…*, 2023, ACM Comp. Surveys

Tabor

- TABOR casts trigger detection as an **optimization problem**: find the trigger that makes the model predict the target class on the test sample

$$\text{argmin}_{\Delta, \mathbf{M}} L(f(\mathbf{x}_t), y_t)$$

$$\mathbf{x}_t = \mathbf{x} \odot (1 - \mathbf{M}) + \Delta \odot \mathbf{M}$$

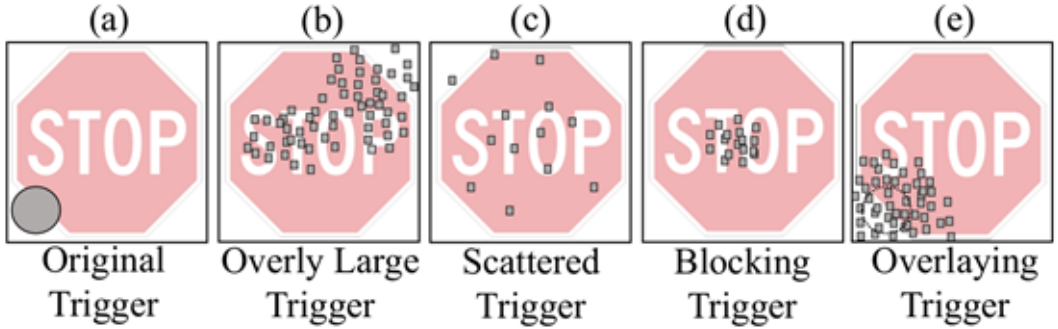


- **M**: mask (trigger shape and location)
- Δ: pattern (trigger color)
- $\mathbf{x_t}$: test sample; $\mathbf{y_t}$: target class
- **M** ⊙ Δ: restored trigger

Guo et al., *TABOR: A Highly Accurate Approach to Inspecting and Restoring Trojan Backdoors AI Systems*, Arxiv 2019

Tabor

- Solving this problem often returns a **wrong trigger** of one of these types:



- TABOR uses **regularization** to penalize these cases and restore the real trigger

Guo et al., *TABOR: A Highly Accurate Approach to Inspecting and Restoring Trojan Backdoors AI Systems*, Arxiv 2019

Tabor

- **Setup**: ImageNet, VGG16; trigger: Firefox logo, top-right corner
- **Precision** / **Recall**: overlap between restored and real trigger; **F1**: harmonic mean

Size	Fidelity Measure					
	Precision		Recall		F1	
	NCleanse	TABOR	NCleanse	TABOR	NCleanse	TABOR
20 × 20	0.061	**0.398**	0.011	**0.168**	0.019	**0.237**
40 × 40	0.664	**0.752**	0.072	**0.197**	0.129	**0.313**
60 × 60	**0.898**	0.779	0.082	**0.118**	0.150	**0.205**
80 × 80	0.312	**0.774**	0.037	**0.141**	0.066	**0.238**
100 × 100	0.902	**0.925**	0.056	**0.105**	0.106	**0.189**

- TABOR restores the trigger **better than Neural Cleanse at every size**

Guo et al., *TABOR: A Highly Accurate Approach to Inspecting and Restoring Trojan Backdoors AI Systems*, Arxiv 2019

Tabor

- **Setup**: GTSRB, CNN; same Firefox-logo trigger
- **Precision** / **Recall**: overlap between restored and real trigger; **F1**: harmonic mean

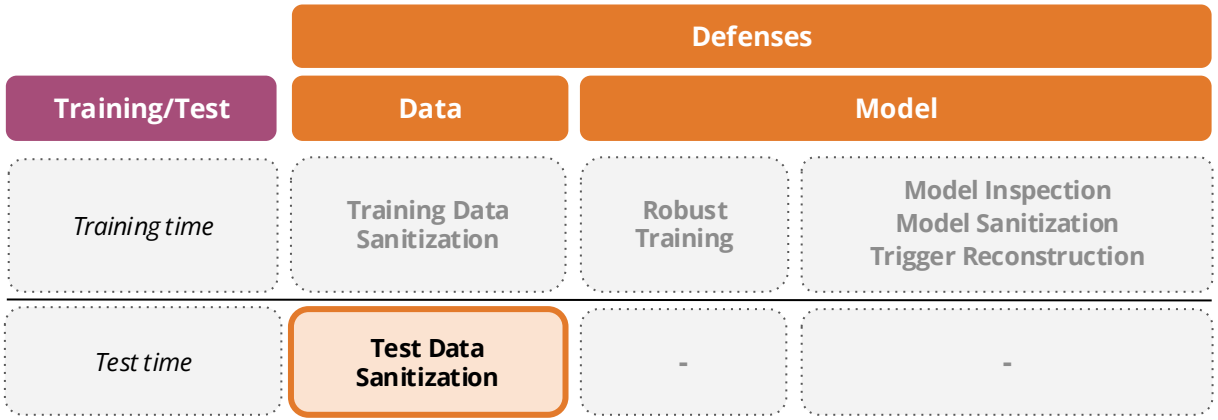
Size	Fidelity Measure					
	Precision		Recall		F1	
	NCleanse	TABOR	NCleanse	TABOR	NCleanse	TABOR
6×6	0.458	**1.000**	0.177	**0.452**	0.256	**0.622**
8×8	0.448	**1.000**	**0.732**	0.285	**0.556**	0.443
10×10	0.882	**0.971**	0.233	**0.337**	0.369	**0.500**
12×12	**0.963**	0.667	0.193	**0.274**	0.321	**0.388**
14×14	0.477	**1.000**	**0.425**	0.275	**0.450**	0.432

- TABOR wins on **4 of 5 trigger sizes**: results depend on dataset, model and trigger

Guo et al., *TABOR: A Highly Accurate Approach to Inspecting and Restoring Trojan Backdoors AI Systems*, Arxiv 2019

Test Data Sanitization

Categorization/Taxonomy of Poisoning Defenses

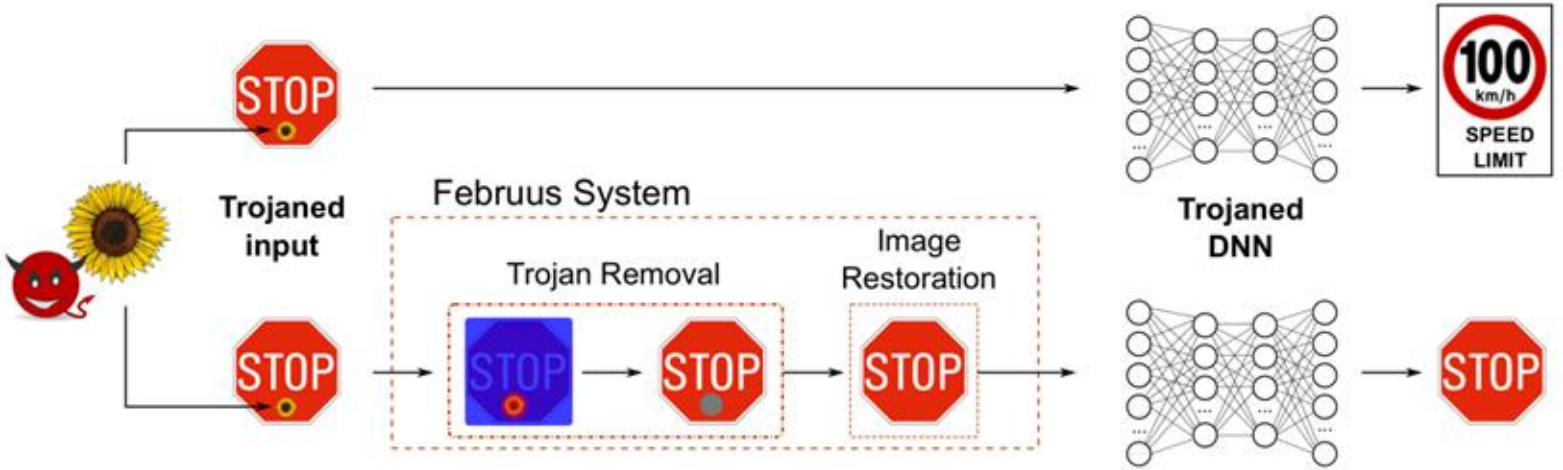


- The defender inspects each test input and **removes the trigger** before classification
- Against **backdoor** poisoning

Cinà,et al., *Wild Pattern Reloaded…*, 2023, ACM Comp. Surveys

Februus

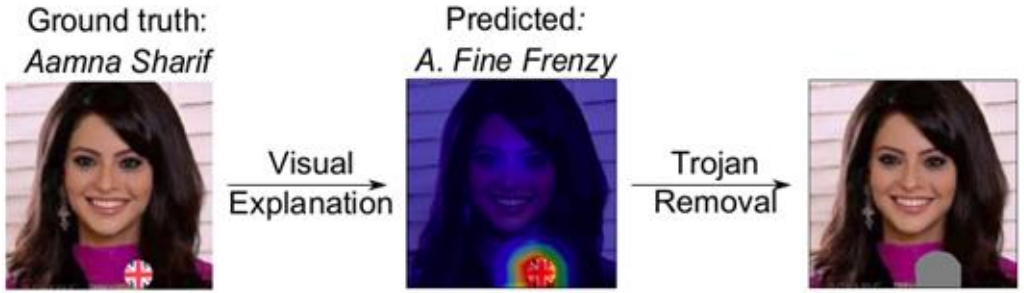
- Februus first **removes the trigger** from the test image, then **restores** the removed region



Doan et al., *Februus: Input Purification Defense Against Trojan Attacks on Deep Neural Network*, ACSAC 2020

Februus

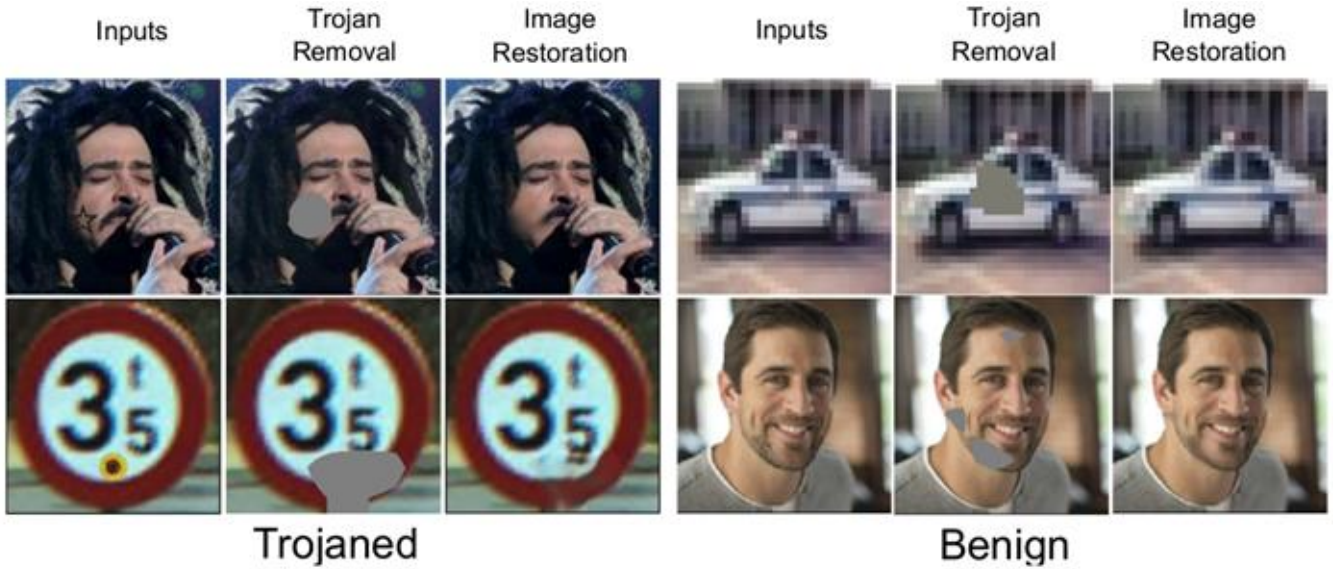
- The **trigger** is located by building a heatmap of the input regions that drive the decision
- A **small region with a very strong contribution** is likely to be the trigger



Doan et al., *Februus: Input Purification Defense Against Trojan Attacks on Deep Neural Network*, ACSAC 2020

Februus

- The removed region is **filled in by a GAN** (generative adversarial network)



Doan et al., *Februus: Input Purification Defense Against Trojan Attacks on Deep Neural Network*, ACSAC 2020

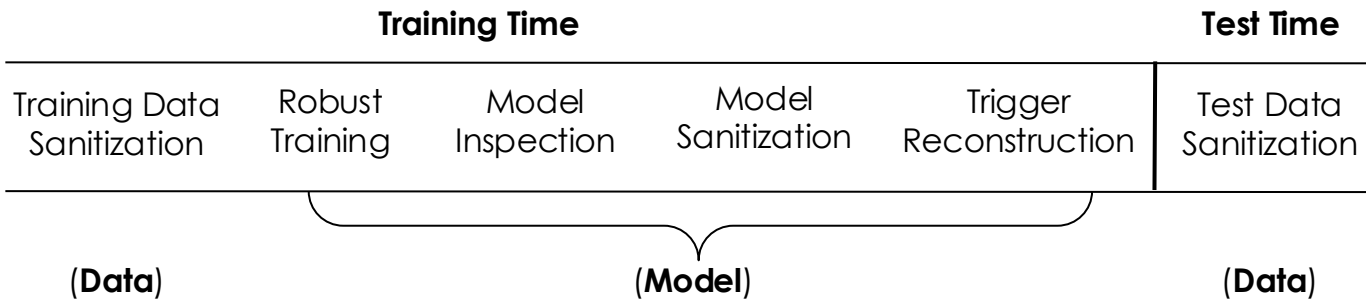
Februus

- **Setup**: CIFAR10 (6 conv + 2 dense layers) and VGGFace2 (VGG-16), trojaned models
- Februus cuts attack success from **100% to ≤ 0.25%**, keeping clean accuracy **within 0.3 points** of a benign model

	Clean accuracy		Attack success rate	
Dataset	**Benign model**	**With Februus**	**Without Februus**	**With Februus**
CIFAR10	90.34%	90.08%	100%	**0.25%**
VGGFace2	91.84%	91.78%	100%	**0.00%**

Doan et al., *Februus: Input Purification Defense Against Trojan Attacks on Deep Neural Network*, ACSAC 2020

Recap & Take Away Messages



Broad coverage
Some defenses protect against several poisoning attacks

No silver bullet
No defense is bulletproof against strong adaptive attacks

Combine defenses
Combining defenses gives higher robustness